

IT4079:NGÔN NGỮ' và PHƯƠNG PHÁP DỊCH

Phạm Đăng Hải
haipd@soict.hust.edu.vn

Chương 3: Phân tích cú pháp

1. Bài toán phân tích cú pháp
2. Phương pháp phân tích cú pháp quay lui
3. Phương pháp phân tích bảng
4. Phương pháp phân tích cú pháp tất định
5. Phân tích cú pháp cho PL/0

Bài toán đặt ra

Cho

- Văn phạm phi ngữ cảnh G

$$G = (V_T, V_N, P, S)$$

- Xâu $\omega \in V_T^*$

Hỏi

- $\omega \in L(G)$?

Nếu $\omega \in L(G)$

```
Program Vidu;  
Begin  
    X := 10  
End.
```

Trong chương trình dịch,
xâu ω là chuỗi các token
thu được từ giai đoạn
trước – phân tích từ vựng

```
PROGRAM IDENT  
SEMICOLON BEGIN IDENT  
ASSIGN NUMBER END  
PERIOD
```

- Chỉ ra các sản xuất đã sử dụng để sinh ra ω
- Cấu trúc nên cây suy dẫn

Phương pháp phân tích

- Kiểm tra xâu phân tích từ trái qua phải
 - Kiểm tra ký hiệu trái nhất của xâu cần phân tích
 - Tới ký hiệu tiếp,.. Cho tới ký hiệu cuối cùng
- Phương pháp xây dựng cây phân tích
 - Trên xuống (*Top-down*): $S \Rightarrow^* \omega?$
 - Dưới lên (*Bottom-up*): $\omega \stackrel{*}{\leftarrow} S?$
- Phương pháp lựa chọn sản xuất ($A \rightarrow \alpha_1 | \dots | \alpha_n$)
 - Quay lui (*backtracking*)
 - Thử lần lượt các sản xuất
 - Tất định (*deterministic*)
 - Xác định được duy nhất một sản xuất thích hợp

Phân tích trái

- **Phân tích trái** của xâu α là dãy các sản xuất được sử dụng trong *suy dẫn trái* từ S ra α
- Các sản xuất được đánh số thứ tự $1, \dots, p$
 - Phân tích là danh sách các số từ 1 đến p
- Ví dụ cho văn phạm

$$1. E \rightarrow T + E$$

$$2. E \rightarrow T$$

$$3. T \rightarrow F * T$$

$$4. T \rightarrow F$$

$$5. F \rightarrow (E)$$

$$6. F \rightarrow a$$

Xét xâu $a^*(a+a)$

$$E \Rightarrow^2 T \Rightarrow^3 F * T \Rightarrow^6 a * T$$

$$\Rightarrow^4 a * F \Rightarrow^5 a * (E)$$

$$\Rightarrow^1 a * (T + E) \Rightarrow^4 a * (F + E)$$

$$\Rightarrow^6 a * (a + E) \Rightarrow^2 a * (a + T)$$

$$\Rightarrow^4 a * (a + F) \Rightarrow^6 a * (a + a)$$

Phân tích phải

- **Phân tích phải** của xâu α là dãy các sản xuất được sử dụng trong *suy dẫn phải* từ S ra α
- Các sản xuất được đánh số thứ tự $1, \dots, p$
 - Phân tích là danh sách các số từ 1 đến p
- Ví dụ cho văn phạm

$$1. E \rightarrow T + E$$

$$2. E \rightarrow T$$

$$3. T \rightarrow F * T$$

$$4. T \rightarrow F$$

$$5. F \rightarrow (E)$$

$$6. F \rightarrow a$$

Xét xâu $a^*(a+a)$

$$E \Rightarrow^2 T \Rightarrow^3 F * T \Rightarrow^4 F * F$$

$$\Rightarrow^5 F * (E) \Rightarrow^1 F * (T + E)$$

$$\Rightarrow^2 F * (T + T) \Rightarrow^4 F * (T + F)$$

$$\Rightarrow^6 F * (T + a) \Rightarrow^4 F * (F + a)$$

$$\Rightarrow^6 F * (a + a) \Rightarrow^6 a^*(a + a)$$

Chương 3: Phân tích cú pháp

1. Bài toán phân tích cú pháp
2. Phương pháp phân tích cú pháp quay lui
3. Phương pháp phân tích bảng
4. Phương pháp phân tích cú pháp tất định
5. Phân tích cú pháp cho PL/0

Giới thiệu

- Tư tưởng chủ yếu của giải thuật
 - Xây dựng cây phân tích cú pháp (cây suy dẫn) cho xâu ω
- Thuật toán **Top-down**
 - Đi từ nút gốc tới nút lá
- Thuật toán **Bottom –up**
 - Quá trình phân tích gọt thu gọn

Phân tích Top-down

Cho VPPNC $G = (V_T, V_N, P, S)$

$\forall$ sản xuất $A \rightarrow \alpha_1 | \dots | \alpha_n$ được đánh số 1, 2, ...

Xây dựng cây phân tích cho xâu ω :

Bước 1: Khởi tạo

- Xây dựng cây chỉ có một nút gốc S
 - S (Start symbol): Ký hiệu khởi đầu
- Gọi **ký hiệu cần phân tích** là ký hiệu đầu tiên của xâu ω
- Gọi S là **nút hoạt động**,

Phân tích Top-down

Bước 2. Tạo các nút con của cây (*một cách đệ quy*)

❖ **Nút hoạt động** là ký hiệu không kết thúc A

- Chọn sản xuất đầu tiên của A chưa được áp dụng: $A \rightarrow X_1 X_2 \dots X_k$ ($k \geq 0$)
- Tạo k con trực tiếp của A với nhãn $X_1, X_2, \dots, X_k$
- Nếu $k > 0$, Lấy X_1 làm nút hoạt động
- Nếu $k = 0$, (*sản xuất* $A \rightarrow \varepsilon$), lấy nút bên phải (*ngay sau*) A là nút hoạt động

Quay lại thực hiện bước 2

Phân tích Top-down

Bước 2. Tạo các nút con của cây (*tiếp*)

❖ **Nút hoạt động** là ký hiệu kết thúc **a**

- So sánh **a** với ký hiệu cần phân tích hiện tại
 - **Nếu trùng nhau**
 - Nút hoạt động là nút bên phải của **a**
 - Ký hiệu cần phân tích là ký hiệu tiếp theo trên xâu vào
 - *Quay lại thực hiện bước 2*
 - **Nếu không trùng nhau**
 - Quay lại bước đã sử dụng một sản xuất và thử sản xuất tiếp.
 - Nếu đã hết khả năng, quay lại bước trước

Phân tích Top-down

3. Điều kiện dừng

- Đã áp dụng hết khả năng mà không tạo được cây $\Rightarrow$ *Xâu không được đoán nhận*
- Tạo ra cây suy dẫn cho xâu vào

Phân tích Top-down

- ❖ Điều kiện áp dụng thuật toán
 - Văn phạm không đệ quy trái
- ❖ Chi phí ($n = l(\omega)$; $C, K > 1$ là các hằng số)
 - Bộ nhớ $C \cdot n$
 - Thời gian K^n

Phân tích Top-down → Ví dụ

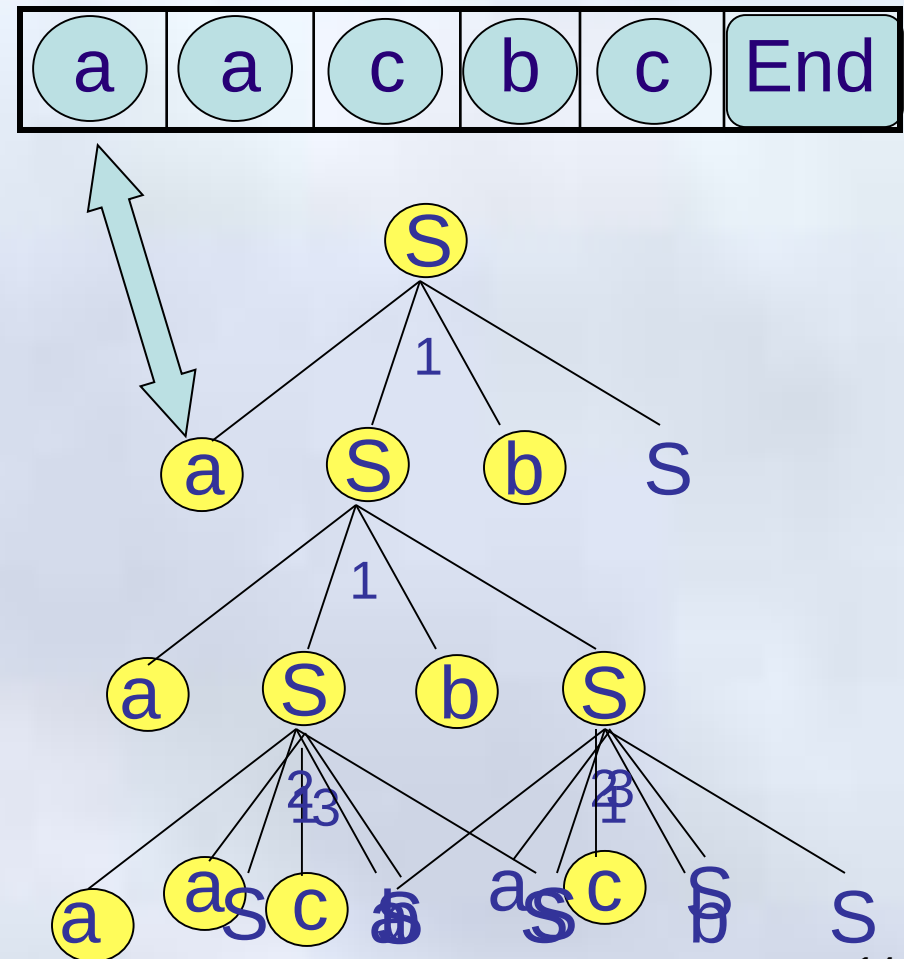
Văn phạm $S \rightarrow aSbS|aS|c$, chuỗi $\omega = aacbc$

Đánh số sản xuất

1. $S \rightarrow aSbS$

2. $S \rightarrow aS$

3. $S \rightarrow c$



Phân tích Top-down → Ví dụ

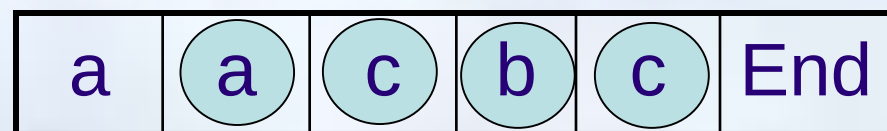
Văn phạm $S \rightarrow aSbS|aS|c$, chuỗi $\omega = aacbc$

Đánh số sản xuất

1. $S \rightarrow aSbS$

2. $S \rightarrow aS$

3. $S \rightarrow c$

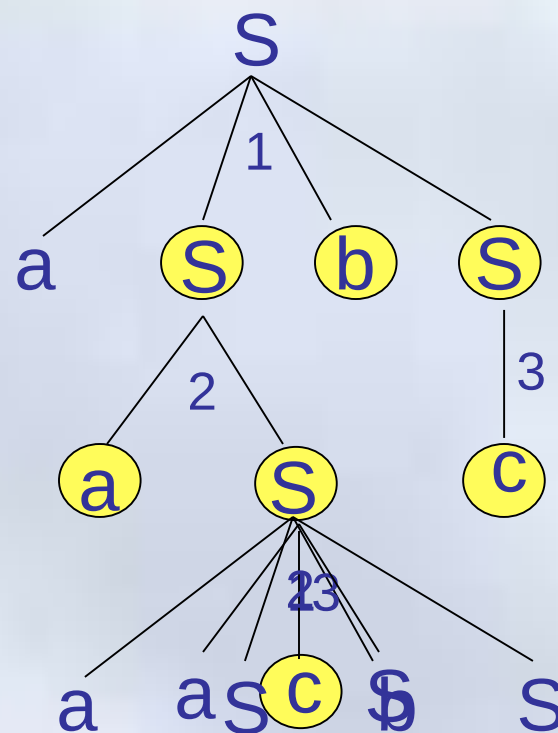


Đánh số sản xuất

1. $S \rightarrow c$

2. $S \rightarrow aS$

3. $S \rightarrow aSbS$



Phân tích Top-down → Bài tập

Cho văn phạm

$$E \rightarrow T + E \mid T$$

$$T \rightarrow F * T \mid F$$

$$F \rightarrow (E) \mid a$$

Xây dựng cây suy dẫn cho xâu $a+a$

Giải thuật phân tích Top-down quay lui (1/8)

Vào

- Văn phạm phi ngữ cảnh không đệ quy trái
- xâu cần phân tích $\omega = a_1 \dots a_n$, $n \geq 0$
- Các sản xuất của G được đánh số $1, \dots, q$

Ra

- Một phân tích trái cho ω (nếu có)
- Thông báo lỗi nếu ngược lại

Quy ước

$\forall A \in V_N$, giả sử $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$

Coi các sản xuất trên là

$A_1 \rightarrow \alpha_1$ $\dots$ $A_n \rightarrow \alpha_n$	}	Để phân biệt sản xuất đã sử dụng của cùng một ký hiệu không kết thúc A
---	---	--

Giải thuật phân tích Top-down quay lui (2/8)

Phương pháp: Dùng 2 stack D_1 và D_2

- D_1 ghi lại những lựa chọn đã sử dụng và những ký hiệu vào đã được phân tích
 - Đầu đọc đã đổi vị trí theo ký hiệu đã phân tích
- D_2 biểu diễn dạng câu trái hiện tại có được bằng cách thay thế các ký hiệu không kết thúc bởi vế phải tương ứng.
 - Đỉnh của D_2 là nút hoạt động của cây tạo ra

Giải thuật phân tích Top-down quay lui (3/8)

Hình trạng của giải thuật

Bộ bốn (s, i, α, β)

- $s \in Q$: Trạng thái hiện thời của giải thuật
 - q : Trạng thái bình thường
 - b : Quay lui
 - t : Kết thúc
- i : Vị trí đầu đọc (*# kết thúc xâu băng vào*)
- α : Nội dung stack thứ nhất, đỉnh ở bên phải
- β : Nội dung stack thứ hai, đỉnh ở bên trái
- Hình trạng ban đầu $(q, 1, \varepsilon, S\#)$

Giải thuật phân tích Top-down quay lui (4/8)

Thực hiện giải thuật

- Bắt đầu từ hình trạng đầu, tính liên tiếp các hình trạng tiếp theo cho đến khi không tính được nữa.
- Ký hiệu $\vdash$ chỉ một thay đổi hình trạng
$$(q, i, \alpha, \beta) \vdash (q', i', \alpha', \beta')$$
- Giải thuật thực hiện theo các dịch chuyển

Giải thuật phân tích Top-down quay lui (5/8)

1. Mở rộng cây

$$(q, i, \alpha, A\beta) \vdash (q, i, \alpha A_1, \gamma_1\beta)$$

$A \in V_N$ và $A_1 \rightarrow \gamma_1$ là lựa chọn đầu tiên của A

2. Phù hợp với ký hiệu vào

$$(q, i, \alpha, a\beta) \vdash (q, i+1, \alpha a, \beta)$$

Ký hiệu $a \in V_T$ ở đỉnh D_2 phù hợp ký hiệu vào

$\Rightarrow$ Chuyển a sang D_1 và dịch chuyển đầu đọc

3. Không phù hợp ký hiệu vào

$$(q, i, \alpha, a\beta) \vdash (b, i, \alpha, a\beta) \quad // \text{chuyển sang}$$

//trạng thái quay lui

Giải thuật phân tích Top-down quay lui (6/8)

4. Quay lui trên sâu vào

$$(b, i, \alpha a, \beta) \mid\!\!\!-\ (b, i-1, \alpha, a\beta) \quad a \in V_T$$

Chuyển ký hiệu trên đỉnh D_1 sang D_2 và dịch đầu đọc sang trái

5. Thử lựa chọn tiếp theo

$$(b, i, \alpha A_j, \gamma_j \beta) \mid\!\!\!-\ (q, i, \alpha A_{j+1}, \gamma_{j+1} \beta)$$

- γ_{j+1} là lựa chọn tiếp của ký hiệu A
- Nếu là sản xuất cuối của A , quay lui tiếp
 - Thay γ_j bởi A và loại bỏ A_j trên đỉnh của D_1

$$(b, i, \alpha A_j, \gamma_j \beta) \mid\!\!\!-\ (b, i, \alpha, A_j \beta)$$

Giải thuật phân tích Top-down quay lui (7/8)

6. Nếu hết khả năng quay lui ($i=1, A \equiv S$):

Không đoán nhận

7. Kết thúc thành công

- $(q, n+1, \alpha, \#) \vdash (t, n+1, \alpha, \varepsilon)$
- Kết luận: Xâu đã được đoán nhận.
- Để tìm phân tích trái, thực hiện hàm $h(\alpha)$

Tìm phân tích trái

- $h(a) = \varepsilon \ \forall a$ là ký hiệu kết thúc
- $h(A_i) = p$, Với p là số hiệu của sản xuất liên hệ với sản xuất $A \rightarrow \gamma$ với γ là lựa chọn thứ i của A

Giải thuật phân tích Top-down quay lui (8/8)

Ví dụ văn phạm

1. $S_1 \rightarrow aSb$

2. $S_2 \rightarrow c$

$\omega: aacbb$

$h(S_1aS_1aS_2cbb)=112$

$(q, 1, \varepsilon, S\#)$

└─ $(q, 1, S_1, aSb\#)$

└─ $(q, 2, S_1a, Sb\#)$

└─ $(q, 2, S_1aS_1, aSbb\#)$

└─ $(q, 3, S_1aS_1a, Sbb\#)$

└─ $(q, 3, S_1aS_1aS_1, aSbbb\#)$

└─ $(b, 3, S_1aS_1aS_1, aSbbb\#)$

└─ $(q, 3, S_1aS_1aS_2, cbb\#)$

└─ $(q, 4, S_1aS_1aS_2c, bb\#)$

└─ $(q, 5, S_1aS_1aS_2cb, b\#)$

└─ $(q, 6, S_1aS_1aS_2cbb, \#)$

└─ $(t, 6, S_1aS_1aS_2cbb, \varepsilon)$

└─ Đoán nhận

Phân tích Bottom-up

- Sử dụng chu trình phân tích phải, thông qua tất cả các suy dẫn phải có thể theo chiều ngược lại phù hợp với xâu vào
 - Là quá trình gặt-thu gọn (**shift – reduce**)
- Thuật toán sử dụng stack S , dùng chứa các ký hiệu của văn phạm đã sinh ra một tiền tố nào đó trên xâu vào

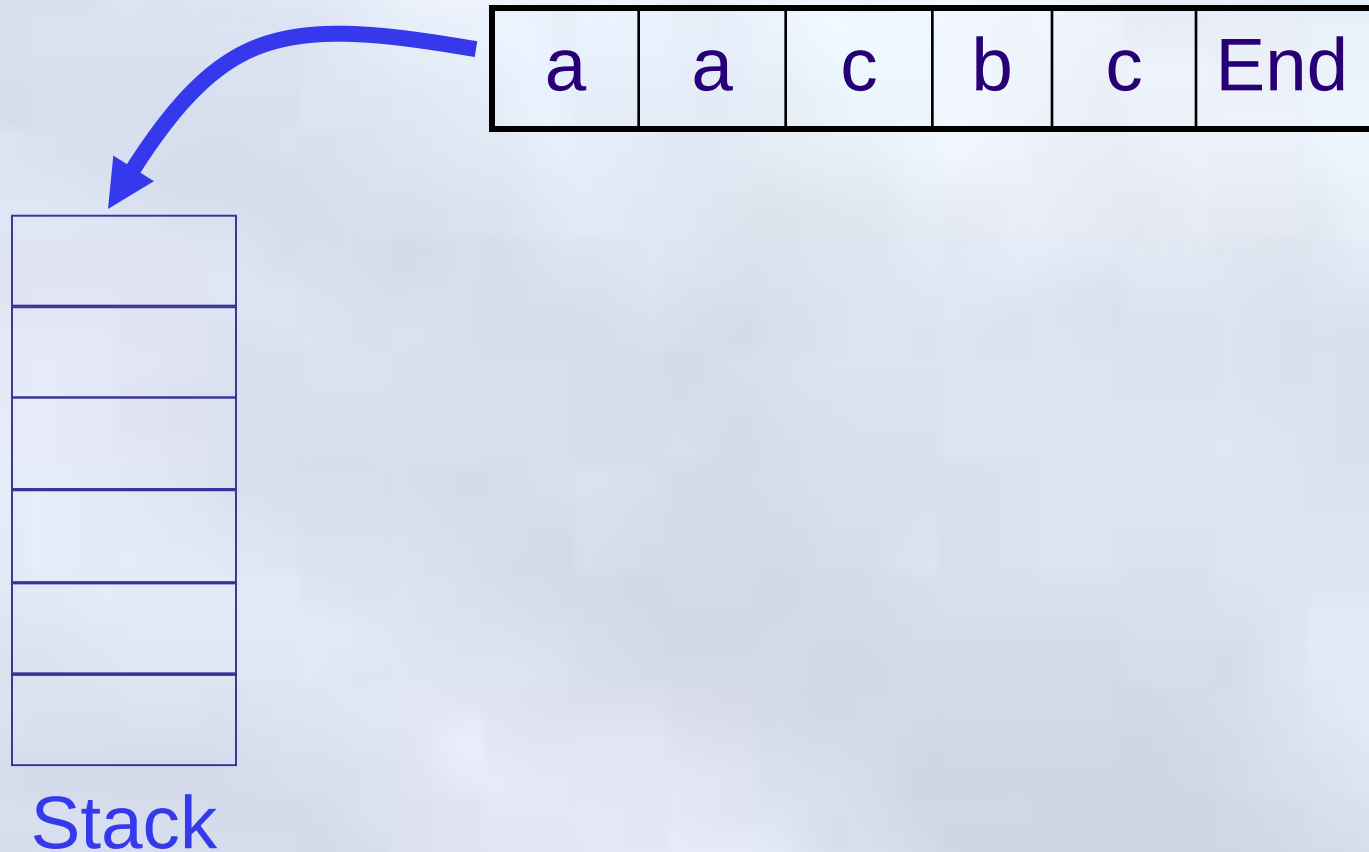
Phân tích Bottom-up

Hoạt động

- Xét tất cả các xâu ω có thể trên đỉnh Stack S
 - Nếu tồn tại một sản xuất $A \rightarrow \omega \in P$, thu gọn xâu ω về A
 - Nếu có nhiều lựa chọn $\rightarrow$ đánh số để thử lần lượt
 - Nếu không thể thu gọn được, gọt ký hiệu tiếp theo của ω vào Stack
 - Nếu đi hết xâu mà không thể thu gọn $\rightarrow$ quay lui lại bước thu gọn sau cùng để thử thu gọn khác
- Thuật toán dừng khi
 - Đã gọt hết các ký hiệu và thu gọn về S
 - Đã thử hết các trường hợp nhưng vẫn không thu gọn

Phân tích Bottom-up $\rightarrow$ Ví dụ

Văn phạm $S \rightarrow aSbS|aS|c$, chuỗi $\omega = aacbc$



Phân tích Bottom-up

❖ Điều kiện áp dụng thuật toán

- Văn phạm không chứa sản xuất dạng

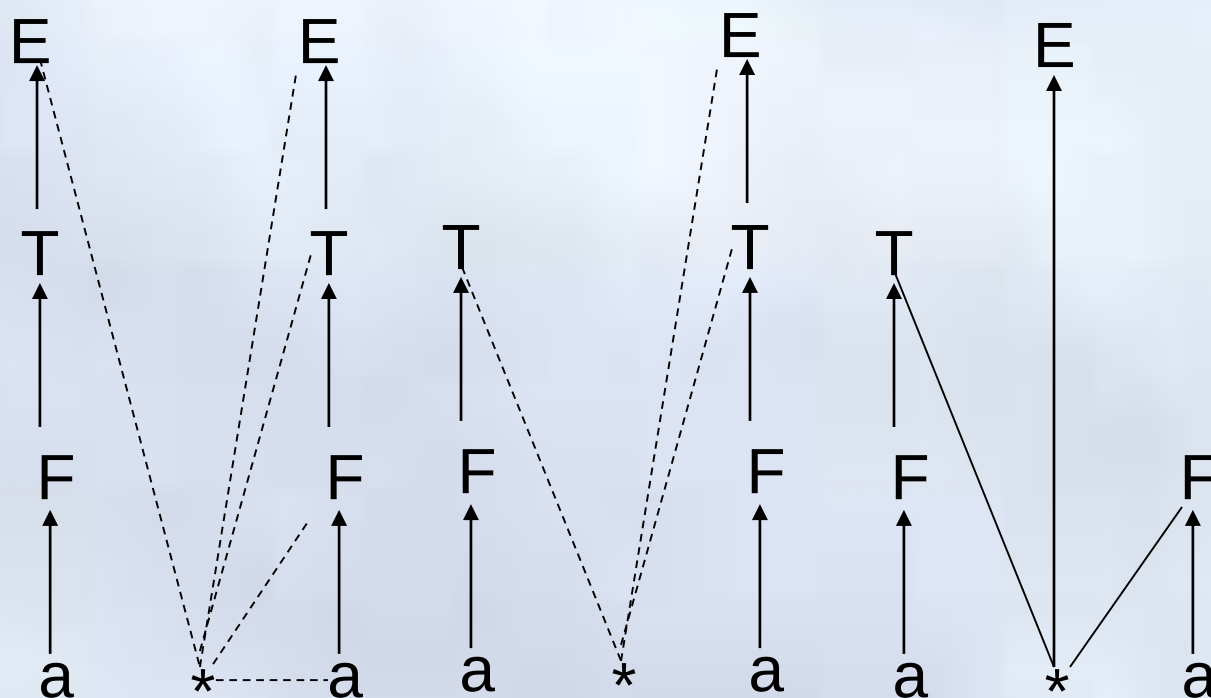
$$A \rightarrow \varepsilon \text{ hoặc } A \Rightarrow^+ A$$

❖ Chi phí ($n = l(\omega)$; $C, K > 1$ là các hằng số)

- Bộ nhớ $C \cdot n$
- Thời gian K^n

Phân tích Bottom-up $\rightarrow$ Ví dụ $E \rightarrow E + T$ $E \rightarrow T$ $T \rightarrow T * F$ $T \rightarrow F$ $F \rightarrow (E)$ $F \rightarrow a$ xâu $\omega = a * a$

Dấu --- thể hiện các xâu trên đỉnh Stack không rút gọn được



Giải thuật phân tích Bottom-Up quay lui

Vào : Văn phạm không chứa sản xuất dạng

$$A \rightarrow \varepsilon \text{ hoặc } A \Rightarrow^+ A$$

Các sản xuất của G được đánh số $1, \dots, q$

Ra: - Phân tích phải cho ω (nếu có)
- Thông báo lỗi nếu ngược lại

Hình trạng của giải thuật: Bộ bốn (s, i, α, β)

- Hình trạng ban đầu $(q, 1, \#, \varepsilon)$
- s : trạng thái thuật toán gồm: $q, b, t; \alpha, \beta$ nội dung các stack
- **Stack D_1** : chứa ký hiệu của văn phạm đã sinh ra tiền tố của xâu vào (*đến vị trí $i-1$*)
- **Stack D_2** : Ghi nhận quá trình gạt (s) thu gọn cần thiết để sinh được D_1 từ xâu vào

Giải thuật phân tích Bottom-Up quay lui

1. Thu gọn

$$(q, i, \alpha\beta, \gamma) \vdash (q, i, \alpha A, j\gamma)$$

- Nếu $A \rightarrow \beta$ là sản xuất thứ j của P
- β là xâu trên đỉnh stack D_1 (được thu gọn về A)

2. Gạt (s:shift)

$$(q, i, \alpha, \gamma) \vdash (q, i+1, \alpha a_i, s\gamma)$$

- Gạt ký hiệu $i+1$ lên đỉnh D_1 , dịch đầu dọc. Đặt s lên D_2 để chỉ ra đã thực hiện thao tác **gạt**
- Nếu $i < n+1$, thực hiện thu gọn
- Nếu $i = n+1$, kiểm tra chấp nhận hoặc thực hiện quay lui

Giải thuật phân tích Bottom-Up quay lui

3. Chấp nhận

$$(q, n+1, \#S, \gamma) \vdash (t, n+1, \#S, \gamma)$$

- Đã đọc hết xâu vào và thu gọn về S.
- Tính phân tích phải bởi thực hiện $h(\alpha)$
 - $h(s) = \varepsilon$
 - $h(j) = j$ với j là số thứ tự sản xuất

4. Quay lui trên xâu vào

Nếu α khác $\#S$ chuyển trạng thái quay lui

$$(q, n+1, \alpha, \gamma) \vdash (b, n+1, \alpha, \gamma)$$

Giải thuật phân tích Bottom-Up quay lui

• Quay lui 1

$$(\mathbf{b}, i, \alpha A, j\gamma) \vdash (\mathbf{q}, i, \alpha'B, k\gamma)$$

- Nếu $A \rightarrow \beta$ là sản xuất thứ j của P
- xâu $\alpha\beta = \alpha'\beta'$ và $B \rightarrow \beta'$ là sản xuất thứ k của P

• Quay lui 2

$$(\mathbf{b}, n+1, \alpha A, j\gamma) \vdash (\mathbf{b}, n+1, \alpha\beta, \gamma)$$

- Nếu $A \rightarrow \beta$ là sản xuất thứ j của P và không có lựa chọn nào khác của $\alpha\beta$ để thu gọn (*như quay lui 1*), thực hiện **tự hủy** thu gọn đã thực hiện
- Nếu đã đọc đến cuối xâu $\rightarrow$ tiếp tục quay lui

Giải thuật phân tích Bottom-Up quay lui

• Quay lui 3

$$(\mathbf{b}, i, \alpha A, j\gamma) \mid\!\!\!\longrightarrow (\mathbf{q}, i+1, \alpha\beta a_i, s\gamma)$$

- Nếu $i < n+1$, $A \rightarrow \beta$ là sản xuất thứ j của P và không còn lựa chọn nào khác của xâu $\alpha\beta$ thì
 - Hủy thu gọn j ($A \rightarrow \beta$),
 - Gạt $\mathbf{a}_i$ vào D_1 và đặt $\mathbf{s}$ vào D_2 rồi tiếp tục thu gọn

• Quay lui 4

$$(b, i, \alpha a, s\gamma) \mid\!\!\!\longrightarrow (b, i-1, \alpha, \gamma)$$

- Nếu đỉnh D_2 là ký hiệu gạt và mọi lựa chọn ở vị trí i đã hết, hoạt động *gạt bị tự hủy*
 - Đầu đọc dịch trái 1 đơn vị
 - Loại bỏ ký hiệu $\mathbf{a}$ khỏi D_1 và $\mathbf{s}$ khỏi D_2

Phân tích Bottom-Up quay lui → Ví dụ

Ví dụ văn phạm

1. $S \rightarrow aSbS$

2. $S \rightarrow aS$

3. $S \rightarrow c$

xâu $\omega = aacbc$

$h(13ss23sss) =$
1323

(q,1, #, ϵ)
|— (q, 2, #a, s) |— (q, 3, #aa,ss)
|— (q, 4, #aac,sss) |— (q, 4, #aaS,3sss)
|— (q, 4, #aS,23sss)
|— (q, 4, #S,223sss)
|— (q, 5, #Sb,s223sss)
|— (q, 6, #Sbc,ss223sss)
|— (q, 6, #SbS,3ss223sss)
|— (**b**, 6, #SbS,3ss223sss)//quay lui
|— (b, 6, #Sbc,ss223sss) //hủy thu gọn
|— (b, 5, #Sb,s223sss) //gạt tự hủy
|— (b, 4, #S,223sss) //gạt tự hủy
|— (**q**, 5, #aSb,s23sss) //quay lui 3
|— (q, 6, #aSbc,ss23sss)
|— (q, 6, #aSbS,3ss23sss)
|— (q, 6, #S,13ss23sss) |— **Chấp nhận**

Phân tích quay lui với PL/0

- Cài đặt phức tạp
- Chi phí thời gian quá lớn nếu chương trình phải phân tích gồm nhiều ký hiệu (từ tổ)
- Không thể thông báo lỗi chi tiết

Chương 3: Phân tích cú pháp

1. Giới thiệu
2. Phương pháp phân tích cú pháp quay lui
3. Phương pháp phân tích bảng
4. Phương pháp phân tích cú pháp tất định
5. Xây dựng bộ phân tích cú pháp cho PL/0

Giới thiệu

- Áp dụng với lớp văn phạm phi ngữ cảnh
- Chi phí: (n : độ dài xâu cần phân tích)
 - n^3 về thời gian
 - n^2 về mặt bộ nhớ
 - Nếu văn phạm đơn nghĩa: n^2 về thời gian
- Phương pháp
 1. Giải thuật CYK (Cocke-Younger-Kasami)
 2. Giải thuật Earley

Giải thuật CYK

- Điều kiện áp dụng
 - Văn phạm phi ngữ cảnh ở dạng chuẩn Chomsky (**CNF**: Chomsky Normal Form)
 - Văn phạm không chứa ε -sản xuất (*Sản xuất dạng $A \rightarrow \varepsilon$*) và sản xuất vô ích
- Nguyên tắc
 - Tạo bảng phân tích tam giác theo xâu cần phân tích

Dạng chuẩn Chomsky

- Khái niệm
 - Văn phạm phi ngữ cảnh được gọi là ở dạng chuẩn Chomsky nếu mọi sản xuất đều có dạng $A \rightarrow BC$ hoặc $A \rightarrow a$ ($A, B, C \in V_N, a \in V_T$)
- Mọi văn phạm không chứa ε -sản xuất đều có thể chuyển về dạng chuẩn Chomsky

Chuyển VPPNC sang dạng chuẩn Chomsky

1. Với mọi sản xuất dạng $A \rightarrow X_1 X_2 \dots X_n$ ($n \geq 2$)

- Nếu $X_i \in V_T$ là ký hiệu kết thúc ($X_i = a$)
 - Thêm ký hiệu $C_a \in V_N$ và sản xuất $C_a \rightarrow a$
 - Thay X_i bởi C_a trong các sản xuất của văn phạm

2. Với mọi sản xuất dạng $A \rightarrow B_1 B_2 \dots B_n$ ($n \geq 3$)

- Thêm các ký hiệu không kết thúc $D_1, D_2, \dots, D_{n-2}$
- Thay sản xuất $A \rightarrow B_1 B_2 \dots B_n$ bởi tập
 - $A \rightarrow B_1 D_1$
 - $D_1 \rightarrow B_2 D_2$
 - $D_{n-2} \rightarrow B_{n-1} B_n$

Dạng chuẩn Chomsky → Ví dụ

$$S \rightarrow bA \mid aB$$

$$A \rightarrow bAA \mid aS \mid a$$

$$B \rightarrow aBB \mid bS \mid b$$

$$S \rightarrow C_b A \mid C_a B$$

$$A \rightarrow C_b D_1 \mid C_a S \mid a$$

$$B \rightarrow C_a D_2 \mid C_b S \mid b$$

$$D_1 \rightarrow AA \quad C_a \rightarrow a$$

$$D_2 \rightarrow BB \quad C_b \rightarrow b$$

Thêm 2 sản xuất

$$C_b \rightarrow b \quad \text{và} \quad C_a \rightarrow a$$

$$S \rightarrow bA : \quad S \rightarrow C_b A$$

$$S \rightarrow aB : \quad S \rightarrow C_a B$$

$$A \rightarrow bAA : \quad A \rightarrow C_b AA$$

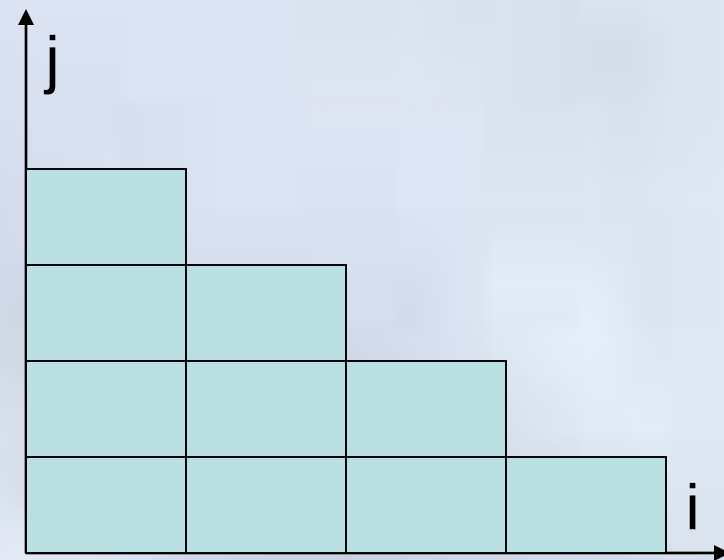
$$A \rightarrow aS : \quad A \rightarrow C_a S$$

$$B \rightarrow aBB : \quad B \rightarrow C_a BB$$

$$B \rightarrow bS : \quad B \rightarrow C_b S$$

Giải thuật CYK → Mô tả bảng

- Bảng dữ liệu T dạng ma trận tam giác dưới, phụ thuộc xâu cần phân tích
- Các phần tử là T_{ij} , i chỉ số cột, j chỉ số hàng
- Mỗi phần tử T_{ij} chứa tập con V_N
- $A \in T_{ij} \Leftrightarrow A \Rightarrow^+ a_i a_{i+1} \dots a_{i+j-1}$
 - j ký hiệu kể từ vị trí i
- Nếu $S \in T_{1n}$
 - $S \Rightarrow^+ a_1 a_2 \dots a_{1+n-1} = \omega$
 - Vậy $\omega \in L(G) \Leftrightarrow S \in T_{1n}$



Giải thuật CYK → Xây dựng bảng

1. Xây dựng các phần tử cho hàng thứ nhất

$$T_{i1} = \{A | A \in V_N \text{ và } A \rightarrow a_i\}$$

2. Giả thiết đã xây dựng xong cho $j-1$ hàng

$$T_{ij} = \{A \mid \exists k \ 1 \leq k < j \text{ thỏa mãn}$$

$$B \in T_{ik}, C \in T_{i+k, j-k} \ A \rightarrow BC \in P \}$$

Rõ ràng: $A \in T_{ij}$ thì

$$A \rightarrow BC \Rightarrow^+ a_i a_{i+1} \dots a_{i+k-1} C \text{ (do } B \in T_{ik})$$

$$\Rightarrow^+ a_i a_{i+1} \dots a_{i+k-1} a_{i+k} a_{i+k+1} \dots a_{i+j-1} = a_i \dots a_{i+j-1}$$

3. Lặp lại bước 2 cho tới khi tính xong các phần tử của T

Giải thuật CYK → Xây dựng bảng → Ví dụ

$$S \rightarrow AA \mid AS \mid b$$

$$A \rightarrow SA \mid AS \mid a$$

$$\omega: abaab$$

	A,S				
	S,A	A,S			
	A,S	S	A,S		
	S,A	A	S	S,A	
	A	S	A	A	S
	a	b	a	a	b
					i
j					

Giải thuật CYK → Tìm cây phân tích

- Là quá trình ngược với xây dựng bảng
- Sử dụng hàm **Gen(i, j, A)** sinh ra dãy phân tích trái ứng với suy dẫn $A \Rightarrow^*_L a_i a_{i+1} \dots a_{i+j-1}$
 - **Gen(1, n, S)** để sinh ra dãy suy dẫn trái của xâu cần phân tích
- **Gen(i, j, A)**
 - $j = 1$: $A \rightarrow a_i$ là sản xuất m. Viết ra giá trị m
 - $j > 1$: Nếu **k** là một số nguyên (nhỏ nhất) thỏa mãn: $B \in T_{ik}, C \in T_{i+k, j-k}, A \rightarrow BC \in P$ là s.xuất m
 - Viết ra giá trị **m**
 - Gọi **Gen(i, k, B)** và **Gen(i+k, j-k, C)**

Giải thuật CYK → Tìm cây phân tích → Ví dụ

1. $S \rightarrow AA$

2. $S \rightarrow AS$

3. $S \rightarrow b$

4. $A \rightarrow SA$

5. $A \rightarrow AS$

6. $A \rightarrow a$

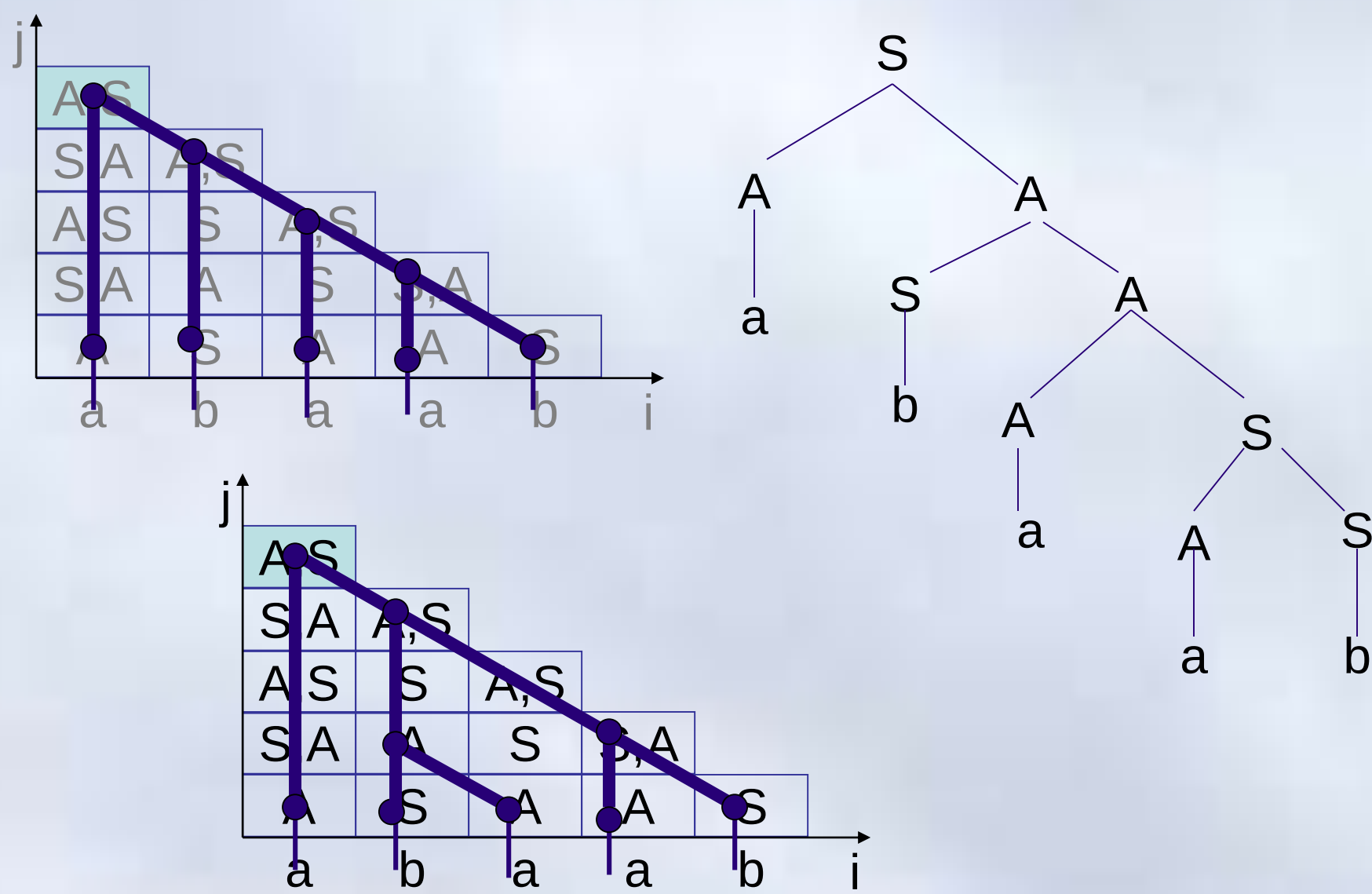
ω : abaab

Gen (1, 5, S)

164356263

262436263

Giải thuật CYK→Tìm cây phân tích



Giải thuật CYK→Bài tập

- $S \rightarrow NP VP$
- $NP \rightarrow Det N \mid NP PP \mid \text{Papa}$
- $VP \rightarrow VP PP \mid V NP$
- $PP \rightarrow P NP$
- $N \rightarrow \text{caviar} \mid \text{spoon}$
- $V \rightarrow \text{ate}$
- $P \rightarrow \text{with}$
- $Det \rightarrow \text{the} \mid \text{a}$

ω : **Papa ate the caviar with a spoon**

Giải thuật CYK → Bài tập

Cho văn phạm $S \rightarrow aSbS \mid aS \mid c$

Dùng phương pháp CYK phân tích $w : aacbc$

Chương 3: Phân tích cú pháp

1. Giới thiệu
2. Phương pháp phân tích cú pháp quay lui
3. Phương pháp phân tích bảng
4. Phương pháp phân tích cú pháp tất định
5. Xây dựng bộ phân tích cú pháp cho PL/0

Nội dung

1. Giới thiệu

2. Phân tích Top-Down

3. Phân tích Bottom-Up

Giới thiệu

Phân tích tất định xem xét một lượt sâu vào từ trái qua phải và tại mỗi bước sẽ xác định được **duy nhất một sản xuất** phù hợp với trạng thái hiện tại

Ví dụ 1

Văn phạm: $S \rightarrow aA$, $A \rightarrow c|bA$

Xâu cần phân tích: abbc

K/hiệu	Xâu phân tích	Sản xuất
S	a bbc	$S \rightarrow aA$
a A	a bbc	So sánh
A	b bc	$A \rightarrow \mathbf{b}A$
A	b c	$A \rightarrow \mathbf{b}A$
A	c	$A \rightarrow c$
c	c	So sánh
-	-	

Sản xuất luôn được lựa
chọn duy nhất nhờ xem
trước một ký hiệu

Ví dụ 2

Văn phạm: $S \rightarrow A|B$, $A \rightarrow aA|c$, $B \rightarrow aB|b$

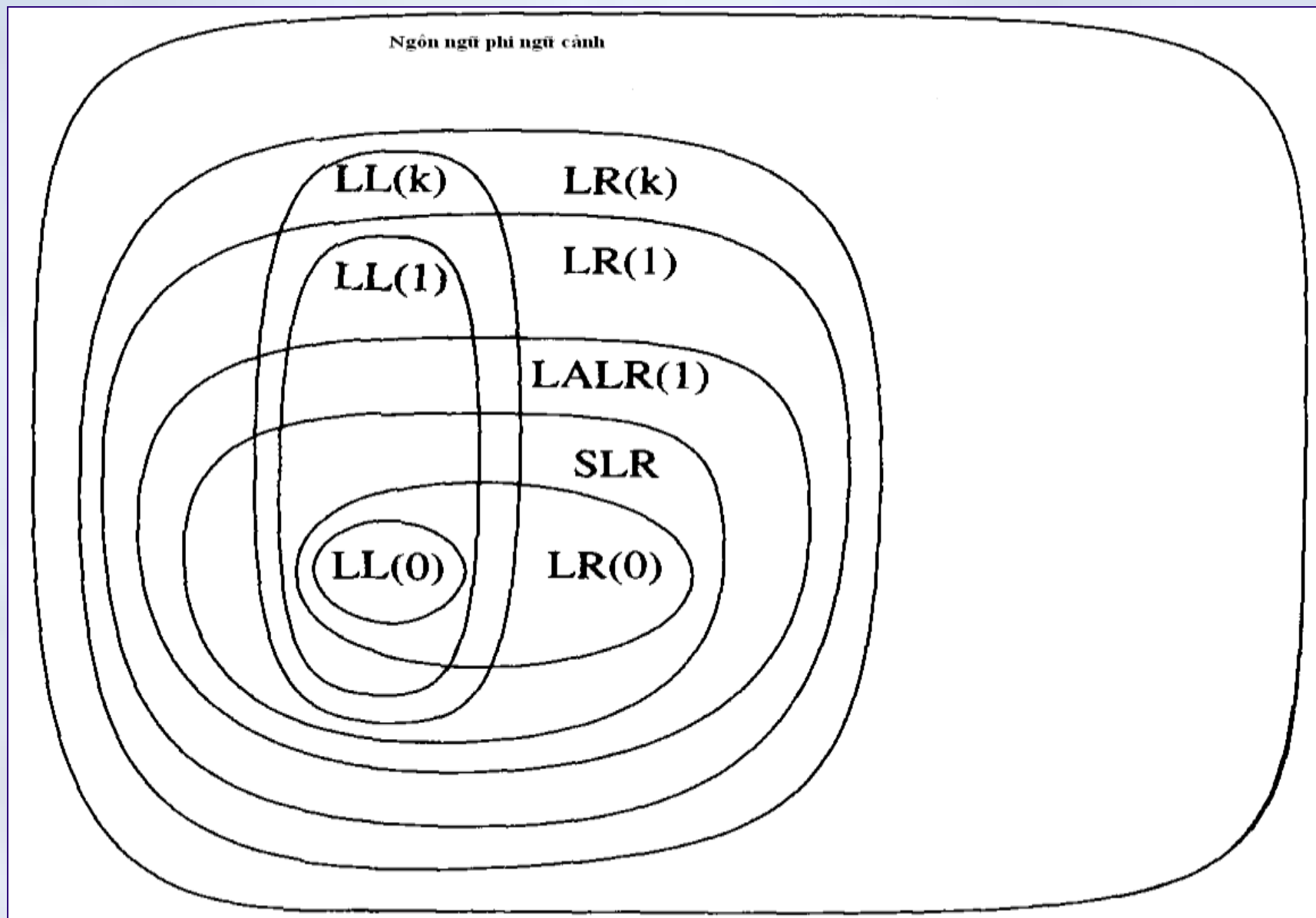
Xâu phân tích: aab

K/hiệu	Xâu phân tích	Sản xuất
S	a ab	$S \rightarrow A$
A	a ab	$A \rightarrow \mathbf{a}A$
a A	a ab	So sánh
A	a b	$A \rightarrow \mathbf{a}A$
a A	a b	So sánh
A	b	?

Nhìn trước một ký hiệu
chưa đủ để lựa chọn
đúng sản xuất $\Rightarrow$ Phải
thực hiện quay lui

Để tránh quay lui, thuật toán phân tích tất định giới hạn trong một số lớp ngôn ngữ: LL(k), LR(k),...

Phân cấp các ngôn ngữ phi ngữ cảnh



Phân tích Top-Down

- Văn phạm LL(k)
- Văn phạm LL(1)
- Phân tích xem trước
- Phân tích đệ quy trên xuống

Ngôn ngữ LL(k)

- Được sinh ra từ văn phạm LL(k)
- Khi thực hiện phân tích văn phạm LL(k), bộ phân tích cần nhìn trước k ký hiệu để quyết định sản xuất nào sẽ được sử dụng
- Để đạt được tính chất này, văn phạm LL(k) cần thỏa mãn một số điều kiện nào đó

LL(k)

→ **k**: Số ký hiệu cần nhìn trước

→ **Left**: Thực hiện các suy dẫn trái nhất

→ **Left**: Câu được phân tích từ trái qua phải

FIRST_k(α)

- Cho văn phạm phi ngữ cảnh $G=(V_T, V_N, P, S)$
- k là một số nguyên dương, $\alpha \in V^*$
- Định nghĩa

$$\begin{aligned}\text{FIRST}_k(\alpha) &= \{x \mid x \in V_T^*, l(x) = k, \alpha \Rightarrow_L^* x\beta\} \\ &= \{x \mid x \in V_T^*, l(x) < k, \alpha \Rightarrow_L^* x\}\end{aligned}$$

FIRST_k(α) là tập các xâu x gồm k ký hiệu kết thúc trái nhất của các xâu được suy dẫn ra từ α

- Chấp nhận trường hợp xâu x không có đủ k ký hiệu nhưng α suy dẫn ra x và không còn ký hiệu nào sau x

$\text{FOLLOW}_k(\alpha)$

- Cho văn phạm phi ngữ cảnh $G=(V_T, V_N, P, S)$
- k là một số nguyên dương, $\alpha \in V^*$
- Định nghĩa

$$\text{FOLLOW}_k(\alpha) = \{x \mid S \Rightarrow_L^* \beta\alpha\delta, x \in \text{FIRST}_k(\delta)\}$$

$\text{FOLLOW}_k(\alpha)$ là tập tất cả các xâu chứa k ký hiệu kết thúc, *đứng ngay đằng sau α* trong bất kỳ dạng câu nào

– Nếu $\alpha \equiv A \in V_N$ và $S \Rightarrow_L^* \beta A$, $\text{FOLLOW}_1(\alpha) \ni \{\varepsilon\}$

Ví dụ

Cho văn phạm

$$S \rightarrow aAb \mid c$$

$$A \rightarrow hS \mid g$$

- $\text{First}_1(S) = \{a, c\}$ $\text{First}_1(A) = \{h, g\}$
- $\text{Follow}_1(S) = \{b\}$ $\text{Follow}_1(A) = \{b\}$
- $\text{First}_2(S) = \{ah, ag, c\}$ $\text{First}_2(A) = \{ha, hc, g\}$
- $\text{Follow}_2(S) = \{b, bb\}$ $\text{Follow}_1(A) = \{b, bb\}$
- $\text{First}_3(S) = \{aha, ahc, agb, c\}$
- $\text{First}_3(A) = \{hah, hag, hc, g\}$
- $\text{Follow}_3(S) = \{b, bb, bbb\}$ $\text{Follow}_3(A) = \{b, bb, bbb\}$

Sản xuất LL(k)

Cho văn phạm phi ngữ cảnh $G = (V_T, V_N, P, S)$,
 một sản xuất từ $A \in V_N$ là sản xuất LL(k) nếu
 thỏa mãn các điều kiện

$$1. S \Rightarrow_L^* xA\alpha \Rightarrow_L x\beta_1\alpha \Rightarrow_L^* xZ_1 \quad (x \in V_T^*)$$

$$2. S \Rightarrow_L^* xA\alpha \Rightarrow_L x\beta_2\alpha \Rightarrow_L^* xZ_2$$

$$3. \text{ Nếu } \text{FIRST}_k(Z_1) = \text{FIRST}_k(Z_2) \text{ thì } \beta_1 = \beta_2$$

Văn phạm LL(k)

- Văn phạm phi ngữ cảnh $G = (V_T, V_N, P, S)$, là LL(k) nếu mọi sản xuất của G đều là sản xuất LL(k)
- Ý nghĩa
 - Nếu G là LL(k) và $xA\alpha$ là một dạng câu, khi đó nếu biết được k ký hiệu kết thúc được suy dẫn ra từ $A\alpha$ thì chỉ *tồn tại duy nhất* một sản xuất từ A thỏa mãn

Ví dụ

Văn phạm $S \rightarrow aAS \mid b$, $A \rightarrow bSa \mid a$
là văn phạm LL(1)

- Xét các sản xuất từ S : $S \rightarrow aAS \mid b$

- Xét các cặp suy dẫn trái

$$S \Rightarrow_L^* xS\alpha \Rightarrow_L x\beta_1\alpha \Rightarrow_L^* xZ_1$$

$$S \Rightarrow_L^* xS\alpha \Rightarrow_L x\beta_2\alpha \Rightarrow_L^* xZ_2$$

- Nếu $FIRST_1(Z_1)=FIRST_1(Z_2)=\{\mathbf{a}\}$ thì từ S phải dùng sản xuất có ký hiệu $\mathbf{a}$ ở đầu: $S \rightarrow \mathbf{a}AS$
- Nếu $FIRST_1(Z_1)=FIRST_1(Z_2)=\{\mathbf{b}\}$ dùng: $S \rightarrow \mathbf{b}$

- Tương tự với các sản xuất từ A : $A \rightarrow bSa \mid a$

Ví dụ

Văn phạm $S \rightarrow aAa \mid bAba$, $A \rightarrow b \mid \varepsilon$

Không phải văn phạm **LL(1)**

Xét sản xuất $S \rightarrow bAba$, nếu đang ở ký hiệu không kết thúc A và nhìn trước **b**, sẽ không biết cần sử dụng sản xuất $A \rightarrow b$ hay $A \rightarrow \varepsilon$

Là văn phạm **LL(2)**

- Nếu đang ở ngữ cảnh **aAa**, và nhìn trước được **ba**, phải áp dụng $A \rightarrow b$, còn nếu nhìn trước được **a#**, phải áp dụng $A \rightarrow \varepsilon$
- Nếu đang ở ngữ cảnh **bAba**, và nhìn trước được **bb**, phải áp dụng $A \rightarrow b$, còn nếu nhìn trước được **ba**, phải áp dụng $A \rightarrow \varepsilon$

Điều kiện để văn phạm là LL(k)

Cho văn phạm phi ngữ cảnh $G = (V_T, V_N, P, S)$,

G là văn phạm LL(k) khi và chỉ khi thỏa mãn

– Nếu $xA\alpha$ là một dạng câu ($S \Rightarrow_L^* xA\alpha$)

$A \rightarrow \beta_1 \in P$ và $A \rightarrow \beta_2 \in P$ ($\beta_1 \neq \beta_2$)

– Thì $\text{FIRST}_k(\beta_1\alpha) \cap \text{FIRST}_k(\beta_2\alpha) = \emptyset$

Văn phạm LL(1)

- Văn phạm LL(k): nhắc tới trong lý thuyết
 - Sử dụng thực tế, $k = 1$.
- Quy ước
 - $\text{FIRST}_1(\alpha) \equiv \text{FIRST}(\alpha)$
 - $\text{FOLLOW}_1(\alpha) \equiv \text{FOLLOW}(\alpha)$
- Định nghĩa lại tập FIRST và FOLLOW
 - Để dễ dàng tính toán
 - Thường quan tâm tới FOLLOW của một ký hiệu không kết thúc

FIRST(α)

$$\begin{aligned}\text{FIRST}(\alpha) &= \{ a \mid a \in V_T, \alpha \Rightarrow_L^* a\beta \} \text{ hoặc} \\ &= \{ \varepsilon \mid \alpha \Rightarrow_L^* \varepsilon \}\end{aligned}$$

Tập tất cả các ký hiệu kết thúc, đứng đầu một chuỗi được suy dẫn ra từ α

FIRST(α) $\rightarrow$ Tính chất

- $a \in V_T^*$ $\text{FIRST}(a) = \{a\}$
- $\text{FIRST}(\varepsilon) = \{\varepsilon\}$
- $A \rightarrow a\gamma$ $\text{FIRST}(A) \supseteq \{a\}$
- $A \rightarrow B\gamma$ $\text{FIRST}(A) \supseteq \text{FIRST}(B) - \{\varepsilon\}$
 - Nếu $b \in \text{FIRST}(B)$ thì $B \Rightarrow_L^* b\beta$
 Vậy $A \rightarrow B\gamma \Rightarrow_L^* b\beta\gamma$. Do đó $\text{FIRST}(A) \supseteq \{b\}$
 - Xét trường hợp $A \rightarrow Ba$ và $B \Rightarrow^* \varepsilon$
 $\text{FIRST}(B) \supseteq \{\varepsilon\}$ nhưng $\{\varepsilon\} \not\subset \text{FIRST}(A)$
- $A \rightarrow B\gamma$ và $B \Rightarrow^* \varepsilon$

$$\text{FIRST}(A) \supseteq (\text{FIRST}(B) - \{\varepsilon\}) \cup \text{FIRST}(\gamma)$$

FOLLOW(A)

$$\text{FOLLOW}(A) = \{ a \mid a \in V_T, S \Rightarrow_L^* \alpha A a \beta \}$$

- Tập tất cả các ký hiệu kết thúc có thể đứng ngay sau ký hiệu không kết thúc A trong một dạng câu nào đó
- Trường hợp đặc biệt, nếu A là ký hiệu bên phải nhất của một dạng câu ($S \Rightarrow_L^* \alpha A$) thì $\text{FOLLOW}(A) \supseteq \{\epsilon\}$ // $\text{FOLLOW}(A) \supseteq \{\#\}$
 - Đánh dấu không có ký hiệu nào ở bên phải A

FOLLOW(A) → Tính chất

- $A \rightarrow \gamma B \alpha$ $\text{FOLLOW}(B) \supseteq \text{FIRST}(\alpha) - \{\epsilon\}$
 - Nếu $\text{FIRST}(\alpha) \supseteq \{a\}$, $\alpha \Rightarrow_L^* a\beta$.
 Vì $A \rightarrow \gamma B \alpha$ nên $A \Rightarrow_L^* \gamma B a\beta$
 Vậy $\text{FOLLOW}(B) \supseteq \{a\}$
 - Xét $S \rightarrow Aa$, $A \rightarrow \gamma B \alpha$ và $\alpha \Rightarrow_L^* \epsilon$
 $\{\epsilon\} \subseteq \text{FIRST}(\alpha)$ nhưng $\{\epsilon\} \not\subseteq \text{FOLLOW}(B)$
- $A \rightarrow \gamma B$ hoặc $A \rightarrow \gamma B \beta$ và $\beta \Rightarrow_L^* \epsilon$
 $\text{FOLLOW}(B) \supseteq \text{FOLLOW}(A)$
 - $\{a\} \subseteq \text{FOLLOW}(A)$ thì $S \Rightarrow_L^* \alpha A a \beta$
 Do $A \rightarrow \gamma B$ nên $S \Rightarrow_L^* \alpha A a \beta \Rightarrow \alpha \gamma B a \beta$

Ví dụ

Cho văn phạm: $E \rightarrow T \mid E+T$

$T \rightarrow F \mid T*F$

$F \rightarrow a \mid (E)$

$\text{FIRST}(F) = \{a, (\}$

$\text{FIRST}(T) = \text{FIRST}(F) = \{a, (\}$

$\text{FIRST}(E) = \text{FIRST}(T) = \{a, (\}$

$\text{FOLLOW}(E) = \{ +,) , \textcolor{red}{\#} \}$

$\text{FOLLOW}(T) = \text{FOLLOW}(T) \cup \{ * \} = \{ +,) , * , \textcolor{red}{\#} \}$

$\text{FOLLOW}(F) = \text{FOLLOW}(T) = \{ +,) , * , \textcolor{red}{\#} \}$

Toán tử $\oplus$

Cho V_1 và V_2 là 2 tập ký hiệu.

Định nghĩa toán tử $\oplus$

$$\begin{aligned} V_1 \oplus V_2 &= V_1 \text{ nếu } \varepsilon \notin V_1 \\ &= (V_1 - \{\varepsilon\}) \cup V_2 \text{ nếu } \varepsilon \in V_1 \end{aligned}$$

Ví dụ

$$\{a, b\} \oplus \{c, d\} = \{a, b\}$$

$$\{\varepsilon, a, b\} \oplus \{c, d\} = \{a, b, c, d\}$$

$$\{\} \oplus \{a, b\} = \{\} \quad // \text{Do tập rỗng không chứa } \varepsilon$$

Bổ đề

$$\text{FIRST}(\alpha\beta) = \text{FIRST}(\alpha) \oplus \text{FIRST}(\beta)$$

$\forall a \neq \epsilon \text{ và } a \in \text{FIRST}(\alpha)$

– Định nghĩa $\oplus$: $\text{FIRST}(\alpha) \oplus \text{FIRST}(\beta) \ni a$

– Định nghĩa FIRST: $\alpha \Rightarrow_L^* a\gamma$

Vậy $\text{FIRST}(\alpha\beta) = \text{FIRST}(a\gamma\beta) \ni a$

$\epsilon \in \text{FIRST}(\alpha)$

– Định nghĩa $\oplus$:

$$\text{FIRST}(\alpha) \oplus \text{FIRST}(\beta) = (\text{FIRST}(\alpha) - \{\epsilon\}) \cup \text{FIRST}(\beta)$$

– Định nghĩa FIRST, $\alpha \Rightarrow_L^* \epsilon$.

• Dựa trên tính chất FIRST ($A \rightarrow B\gamma$ và $B \Rightarrow^* \epsilon$)

$$\text{FIRST}(\alpha\beta) = (\text{FIRST}(\alpha) - \{\epsilon\}) \cup \text{FIRST}(\beta)$$

Tính toán $\text{FIRST}(\alpha)$

Nếu $\alpha = X_1X_2..X_n$ với $X_i \in V$

$$\text{FIRST}(\alpha) = \text{FIRST}(X_1) \oplus \\ \text{FIRST}(X_2) \oplus \dots \oplus \text{FIRST}(X_n)$$

Thuật toán tính $\text{FIRST}(A)$, $A \in V_N$

Xây dựng các tập $F_i(X)$ với $X \in V$

1. $a \in V_T$ $F_i(a) = \{a\} \forall i$
2. $F_0(A) = \{a | a \in V_T \text{ và } A \rightarrow a\alpha \in P\} \cup \{\varepsilon | A \rightarrow \varepsilon \in P\}$
3. Giả thiết đã tính được tập $F_0, F_1, \dots, F_{i-1}$, tính F_i
 - $F_i(A) = \{a | a \in V_T, \text{ Nếu } A \rightarrow X_1 X_2 \dots X_n \in P, \text{ thì}$
 $a \in F_{i-1}(X_1) \oplus F_{i-1}(X_2) \oplus \dots \oplus F_{i-1}(X_n)$
4. Thuật toán dừng khi các F_i không đổi (luôn đúng vì $F_{i-1}(X) \subseteq F_i(X) \subseteq V_T$).

Ví dụ tính FIRST

$$E \rightarrow T \mid E+T$$

$$T \rightarrow F \mid T^*F$$

$$F \rightarrow a \mid (E)$$

X	F ₀ (X)	F ₁ (X)	F ₂ (X)	F ₃ (X)	
E	∅	∅	a, (	a, (	Không đổi
T	∅	a, (	a, (	Không đổi	
F	a, (	a, (	Không đổi		

Thuật toán tính FOLLOW(A)

Thực hiện các quy tắc sau cho tới khi không xuất hiện các thay đổi trong tập FOLLOW

1. Đặt # vào tập FOLLOW(S)

- S là ký hiệu khởi đầu
- $\# \notin V$, dùng đánh dấu kết thúc chuỗi cần phân tích

2. Với mọi sản xuất có dạng $A \rightarrow \alpha B \beta$, thêm $\text{FIRST}(\beta) - \{\epsilon\}$ vào FOLLOW(B)

3. Nếu tồn tại sản xuất $A \rightarrow \alpha B$ hoặc có $A \rightarrow \alpha B \beta$ mà $\{\epsilon\} \subseteq \text{FIRST}(\beta)$ thì thêm FOLLOW(A) vào FOLLOW(B)

Ví dụ tính FOLLOW

VP: $E \rightarrow T \mid E+T, \quad T \rightarrow F \mid T*F, \quad F \rightarrow a \mid (E)$

Quy tắc	Sản xuất áp dụng	FOLLOW		
		E	T	F
1	Khởi tạo	#	$\emptyset$	$\emptyset$
2	$E \rightarrow E+T$	+, #		
	$T \rightarrow T*F$		*	
	$F \rightarrow (E)$	), +, #		
3	$E \rightarrow E+T$ $E \rightarrow T$		*,), +, #	
	$T \rightarrow T*F$ $T \rightarrow F$			*,), +, #
FOLLOW (X)		), +, #	*,), +, #	*,), +, #

Ví dụ

$$S \rightarrow BA$$

$$A \rightarrow + BA \mid \varepsilon$$

$$B \rightarrow DC$$

$$C \rightarrow * DC \mid \varepsilon$$

$$D \rightarrow (S) \mid a$$

Tính $\text{FIRST}(X)$ và $\text{FOLLOW}(X)$ với $X \in V_N$

Ví dụ tính FIRST

X	$F_0(X)$	$F_1(X)$	$F_2(X)$	$F_3(X)$	FIRST
S	$\emptyset$	$\emptyset$	(, a	(, a	{ (, a }
A	+, ϵ	+, ϵ			{ +, ϵ }
B	$\emptyset$	(, a	(, a		{ (, a }
C	*, ϵ	*, ϵ			{ *, ϵ }
D	(, a	(, a			{ (, a }

Ví dụ tính FOLLOW

Quy tắc	Sản xuất áp dụng	FOLLOW				
		S	A	B	C	D
1	Khởi tạo	#				
2	$S \rightarrow BA, A \rightarrow +BA$			+		
	$B \rightarrow DC, C \rightarrow *DC$					*
	$D \rightarrow (S)$	)				
3	$S \rightarrow BA$		#,)			
	$A \rightarrow +BA, A \rightarrow \epsilon$			#,)		
	$B \rightarrow DC$				#, +,)	
	$C \rightarrow *DC, C \rightarrow \epsilon$					#, +,)
FOLLOW(X)		#,)	#,)	#, +,)	#, +,)	*, #, +,)

Điều kiện để văn phạm là LL(1)

Văn phạm phi ngữ cảnh $G = (V_T, V_N, P, S)$ là LL(1) nếu thỏa mãn các điều kiện sau

- Mọi sản xuất dạng $A \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_n$, $n \geq 2$ thỏa mãn $\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) = \emptyset$, $i \neq j$

Ý nghĩa: Nếu $A \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_n$ thì các α_i phải bắt đầu bởi các ký hiệu kết thúc khác nhau

- $\forall A \in V_N$, $A \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_n$ và $\exists \alpha_j \Rightarrow * \varepsilon$ thì

$$\text{FIRST}(\alpha_i) \cap \text{FOLLOW}(A) = \emptyset \quad \forall i \neq j$$

Ý nghĩa: Nếu $A \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_n | \varepsilon$ và $b \in \text{FOLLOW}(A)$ thì không có nhánh α_i nào được bắt đầu bởi ký hiệu b

Văn phạm là LL(1) đơn giản

Văn phạm phi ngữ cảnh $G = (V_T, V_N, P, S)$ là LL(1) đơn giản nếu mọi sản xuất của văn phạm có dạng

$$A \rightarrow a_1 \alpha_1 \mid a_2 \alpha_2 \mid \dots \mid a_n \alpha_n$$

Trong đó $a_i \in V_T$ và $a_i \neq a_j$ với $i \neq j$

Ví dụ về văn phạm LL(1)

$$1. E \rightarrow T \mid E+T$$

$$2. T \rightarrow F \mid T^*F$$

$$3. F \rightarrow a \mid (E)$$

- Sản xuất (3) là sản xuất LL(1) đơn giản
- Xét sản xuất (1)
 - $\text{FIRST}(T) = \{ a, (\}$
 - $\text{FIRST}(E+T) = \text{FIRST}(E) = \{ a, (\}$
 - Vậy $\text{FIRST}(T) \cap \text{FIRST}(E+T) = \{ a, (\} \neq \emptyset$

$\Rightarrow$ Văn phạm không phải là LL(1)

Ví dụ: Xét $a + a / a^* a$. Chỉ đọc được a không chọn đc SX

Ví dụ về văn phạm LL(1)

1. $S \rightarrow BA$
2. $A \rightarrow + BA \mid \varepsilon$
3. $B \rightarrow DC$
4. $C \rightarrow * DC \mid \varepsilon$
5. $D \rightarrow (S) \mid a$

- Sản xuất (1) & (3) là các sản xuất đơn
- Sản xuất (5) là sản xuất LL(1) đơn giản
- Sản xuất (2)
 - $\text{FIRST}(+BA) \cap \text{FOLLOW}(A) = \{+\} \cap \{\#,)\} = \emptyset$
- Sản xuất (4)
 - $\text{FIRST}(*DC) \cap \text{FOLLOW}(C) = \{*\} \cap \{+, \#,)\} = \emptyset$

Khử đệ quy trái

- Các văn phạm LL(1) không đệ quy trái
 - Có thể khử đệ quy trái của VPPNC
- Đệ quy trái (*trực tiếp*)

Sản xuất: $A \rightarrow A\alpha_1 \mid \dots \mid A\alpha_n \mid \beta_1 \mid \dots \mid \beta_m$ với $\beta_i[1] \neq A$

- Đệ quy trái dừng khi áp dụng $SX A \rightarrow \beta_1 \mid \dots \mid \beta_m$
- Từ A sinh ra các xâu dạng

$$(\beta_1 \mid \dots \mid \beta_m) (\alpha_1 \mid \dots \mid \alpha_n)^*$$

- Xâu trên được sinh ra từ văn phạm
 - $A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_m A'$
 - $A' \rightarrow \varepsilon \mid \alpha_1 A' \mid \dots \mid \alpha_n A'$

Khử đệ quy trái

- Ví dụ:
 - Văn phạm $A \rightarrow Aa \mid b$ đệ quy trái trực tiếp
 - Tương đương với văn phạm không đệ quy trái
$$A \rightarrow bB; B \rightarrow \varepsilon \mid aB$$
 - Ngôn ngữ được sinh ra bởi 2 VP: ba^*
- Ghi chú
 - Loại bỏ đệ quy trái $\Rightarrow$ cây phân tích bị thay đổi
 - Cần phải cấu trúc lại cây phân tích để có được cấu trúc gốc

Khử đệ quy trái → Ví dụ

$$1. E \rightarrow T \mid E+T$$

$$2. T \rightarrow F \mid T * F$$

$$3. F \rightarrow a \mid (E)$$

$$E \rightarrow T \mid E+T$$

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \varepsilon$$

$$T \rightarrow F \mid T * F$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \varepsilon$$

$$S \rightarrow BA$$

$$A \rightarrow +BA \mid \varepsilon$$

$$B \rightarrow DC$$

$$C \rightarrow *DC \mid \varepsilon$$

$$D \rightarrow (S) \mid a$$

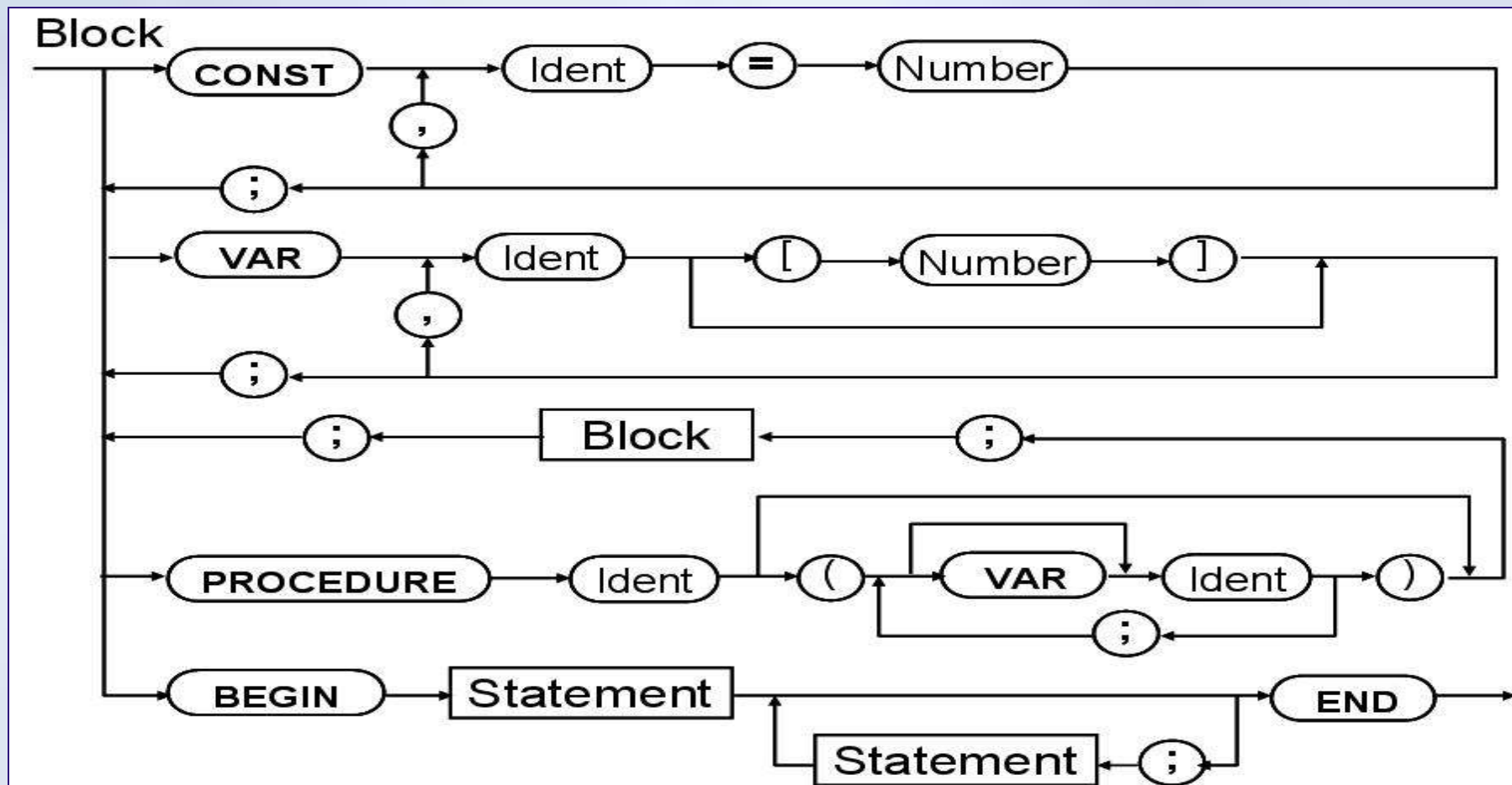
Văn phạm là LL(1) trên sơ đồ cú pháp

- Ở mỗi lối rẽ, các nhánh phải bắt đầu bằng các ký hiệu khác nhau
- Nếu sơ đồ có chứa một đường rỗng (*statement*) thì mọi ký hiệu đứng sau ký hiệu được biểu diễn bởi sơ đồ phải khác các ký hiệu đứng đầu các nhánh của sơ đồ

Văn phạm PL/0 là LL(1) ?

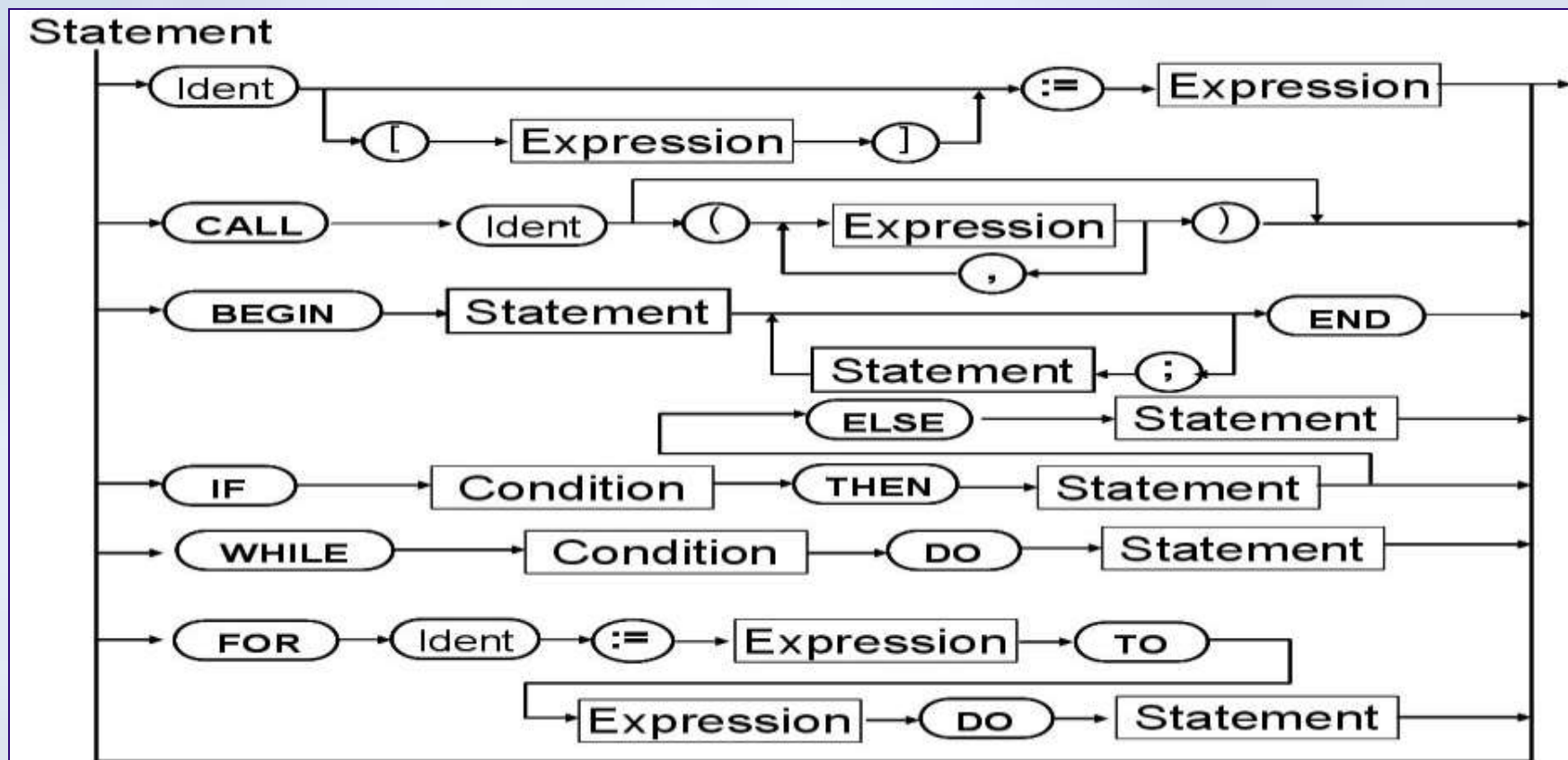
A	FIRST(A)	FOLLOW(A)
Block	CONST VAR PROCEDURE BEGIN	. ;
Statement	Ident CALL BEGIN IF WHILE FOR, ε	; END
Expression	+ - (ident number	; END TO THEN DO) -] < <= > >= = !=
Condition	+ - (ident number, ODD	THEN DO
Term	Ident number (	; END TO THEN DO) - + < <= > >= = !=]
Factor	Ident number (	; END TO THEN DO + - * /) < <= > >= = !=]

Văn phạm PL/0 là LL(1) ?



Các nhánh của sơ đồ bắt đầu bởi một từ khóa (từ tổ) khác nhau (CONST, VAR, PROCEDURE, BEGIN). Vậy <Block> là một sản xuất LL(1) đơn giản

Văn phạm PL/0 là LL(1) ?



Điều kiện 1: Các nhánh bắt đầu bởi các từ tổ khác nhau

Điều kiện 2: (*Thỏa mãn*)

$FOLLOW(<Statement>) = \{END, ., ;\}$

$FIRST(<Statement>) = \{Ident, CALL, BEGIN, IF, WHILE, FOR\}$

Thuật toán phân tích xem trước

- Chỉ nghiên cứu với văn phạm LL(1)
 - Cho phép phân tích đúng nhờ đọc trước 1 ký hiệu trên đầu vào
- Bao gồm 2 giai đoạn
 - Xây dựng ma trận phân tích
 - Thực hiện phân tích xem trước

Xây dựng ma trận phân tích

Xây dựng ma trận phân tích M trên tập:

$$(V \cup \{\#\}) \times (V_T \cup \{\#\})$$

1. Nếu $A \rightarrow \alpha$ là sản xuất thứ k của P

- Nếu $\forall a \neq \epsilon$ và $a \in \text{FIRST}(\alpha)$ thì $M[A, a] = (\alpha, k)$
- Nếu $\epsilon \in \text{FIRST}(\alpha)$ thì

$$\forall b \in \text{FOLLOW}(A), M[A, b] = (\alpha, k)$$

2. $M[a, a] = \text{“Đẩy”}$

3. $M[\#, \#] = \text{“Nhận”}$

4. Mọi ô khác có giá trị sai

Xây dựng ma trận phân tích $\rightarrow$ Ví dụ

1. $S \rightarrow aAS$

2. $S \rightarrow b$

3. $A \rightarrow a$

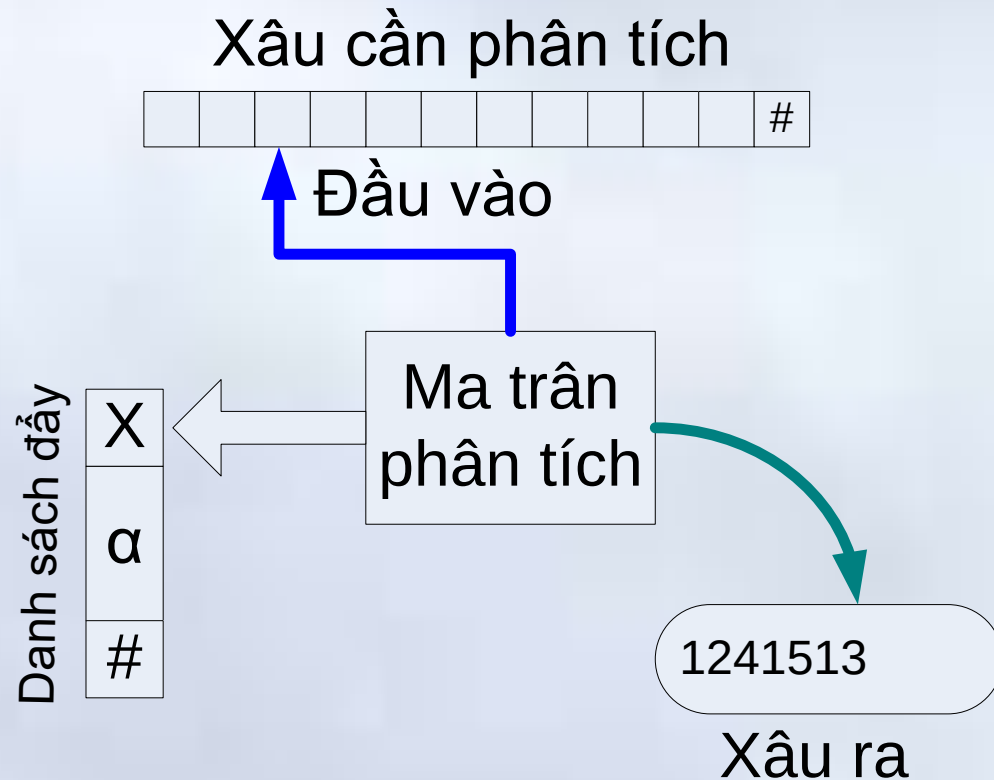
4. $A \rightarrow bSA$

M	a	b	#
S	aAS, 1	b, 2	Sai
A	a, 3	bSA, 4	Sai
a	Đẩy	Sai	Sai
b	Sai	Đẩy	Sai
#	Sai	Sai	Nhận

Phân tích xem trước → Mô tả

Thuật toán gồm

- **Xâu cần phân tích**
 - Đọc bởi một đầu vào
- **Ma trận phân tích M**
- **Danh sách đẩy**
 - Chứa xâu $X\alpha\#$
 - $\#$ Là đáy danh sách
 - $X\alpha \in V^*$
- **Xâu ra**
 - Viết bởi một đầu ghi
 - Chứa số thứ tự các SX



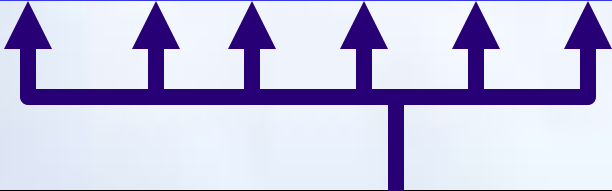
Phân tích xem trước $\rightarrow$ Hoạt động

Thuật toán dựa vào ma trận phân tích M, ký hiệu đọc được bởi đầu đọc và ký hiệu trên đỉnh Stack để quyết định công việc

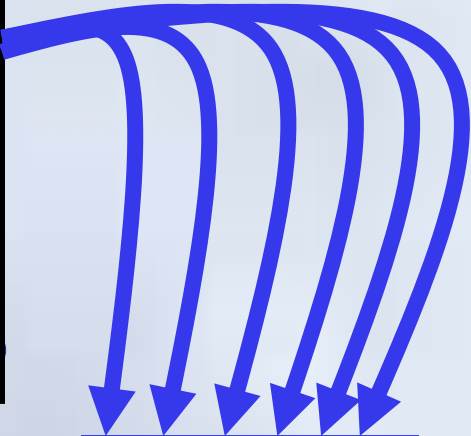
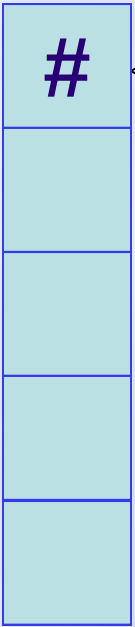
- $M[X,a] = \{\beta, k\}$: X trên đỉnh DS được thay bằng xâu β (ký hiệu đầu của β ở trên); Viết k ra xâu ra
- $M[X,a] = \text{“đẩy”}$: Đầu đọc dịch phải một đơn vị, X bị loại bỏ khỏi danh sách
- $M[X,a] = \text{“sai”}$: Dừng lại và thông báo xâu không được đoán nhận
- $M[X,a] = \text{“nhận”}$: Thuật toán dừng, xâu được đoán nhận và xâu ra chứa DS các SX đã dùng

Phân tích xem trước → Ví dụ

a b b a b #



M	a	b	#
S	aAS,1	b, 2	Sai
A	a, 3	bSA,4	Sai
a	Đẩy	Sai	Sai
b	Sai	Đẩy	Sai
#	Sai	Sai	Nhận



14232

Xâu abbab được đoán nhận
với các sản xuất 14232

Phân tích xem trước $\rightarrow$ Hình trạng thuật toán

- Hình trạng của thuật toán là bộ 3 $(x\#, \alpha\#, \pi)$
 - $x \in V_T^*$ là phần chưa xét của xâu vào
 - $\#$ là ký hiệu đánh dấu kết thúc xâu cần phân tích
 - $\alpha \in V$ là nội dung danh sách đẩy
 - $\#$ là ký hiệu nằm dưới của danh sách đẩy. $\# \notin V$
 - π nội dung của xâu ra
- Hình trạng ban đầu $(\omega\#, S\#, \varepsilon)$
 - ω là xâu cần phân tích
- Hình trạng cuối nếu thành công $(\#, \#, \pi)$
 - π là danh sách các sản xuất đã sử dụng

Phân tích xem trước $\rightarrow$ Ví dụ

Xét chuỗi $\omega \equiv \text{abbab}$

(**a**bbab#, **S**#, ϵ)

$\rightarrow$ (**a**bbab#, **a**AS#,1) $\rightarrow^d$ (**b**bab#, **A**S#,1)

$\rightarrow$ (**b**bab#, **b**SAS#,14) $\rightarrow^d$ (**b**ab#, **S**AS#,14)

$\rightarrow$ (**b**ab#, **b**AS#,142) $\rightarrow^d$ (**a**b#, **A**S#,142)

$\rightarrow$ (**a**b#, **a**S#,1423) $\rightarrow^d$ (**b**#, **S**#,1423)

$\rightarrow$ (**b**#, **b**#,14232) $\rightarrow^d$ (**#**, **#**,14232)

$\rightarrow$ Nhận

Phân tích xem trước $\rightarrow$ Ví dụ 2

Cho văn phạm

- | | |
|--------------------------------|--|
| 1. $S \rightarrow BA$ | $\text{FIRST}(S) = \text{FIRST}(B) = \text{FIRST}(D) = \{ (, a \}$ |
| 2. $A \rightarrow +BA$ | $\text{FIRST}(A) = \{ +, \varepsilon \}$ |
| 3. $A \rightarrow \varepsilon$ | $\text{FIRST}(C) = \{ *, \varepsilon \}$ |
| 4. $B \rightarrow DC$ | $\text{FOLLOW}(S) = \text{FOLLOW}(A) = \{ \#,) \}$ |
| 5. $C \rightarrow ^*DC$ | $\text{FOLLOW}(B) = \text{FOLLOW}(C) = \{ +, \#,) \}$ |
| 6. $C \rightarrow \varepsilon$ | $\text{FOLLOW}(D) = \{ *, +, \#,) \}$ |
| 7. $D \rightarrow (S)$ | Văn phạm đã cho là LL(1) |
| 8. $D \rightarrow a$ | |

Ví dụ 2→Ma trận phân tích

M	a	(	)	+	*	#
S	BA, 1	BA,1				
A			$\epsilon, 3$	+BA,2		$\epsilon, 3$
B	DC,4	DC,4				
C			$\epsilon, 6$	$\epsilon, 6$	*DC,5	$\epsilon, 6$
D	a,8	(S),7				
a	Đẩy					
(		Đẩy				
)			Đẩy			
+				Đẩy		
*					Đẩy	
#						Nhận

Ví dụ 2 → Phân tích xâu (a^*a)
$$[(a^*a)\#, S\#, \epsilon]$$

$\rightarrow [(a^*a)\#, BA\#, 1]$ $\rightarrow [(a^*a)\#, DCA\#, 14]$
 $\rightarrow [(a^*a)\#, (S)CA\#, 147]$ $\rightarrow^d [a^*a)\#, S)CA\#, 147]$
 $\rightarrow [a^*a)\#, BA)CA\#, 1471]$ $\rightarrow [a^*a)\#, DCA)CA\#, 14714]$
 $\rightarrow [a^*a)\#, aCA)CA\#, 147148]$
 $\rightarrow^d [*a)\#, CA)CA\#, 147148]$
 $\rightarrow [*a)\#, *DCA)CA\#, 1471485]$
 $\rightarrow^d [a)\#, DCA)CA\#, 1471485]$
 $\rightarrow [a)\#, aCA)CA\#, 14714858]$
 $\rightarrow^d [)\#, CA)CA\#, 14714858]$ $\rightarrow [)\#, A)CA\#, 147148586]$
 $\rightarrow [)\#,)CA\#, 1471485863]$ $\rightarrow^d [\#, CA\#, 1471485863]$
 $\rightarrow [\#, A\#, 14714858636]$ $\rightarrow [\#, \#, 147148586363]$
 $\rightarrow \text{Nhận}$

Phương pháp đệ quy trên xuống

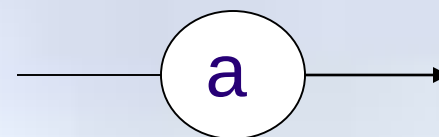
- Sử dụng để phân tích cú pháp cho các văn phạm LL(1)
- Có thể mở rộng cho văn phạm LL(k), nhưng việc tính toán phức tạp
- Sử dụng để phân tích văn phạm khác có thể dẫn đến lặp vô hạn
- Bộ phân tích gồm một tập thủ tục, mỗi thủ tục ứng với một sơ đồ cú pháp (*một ký hiệu không kết thúc*)

Phương pháp đệ quy trên xuống

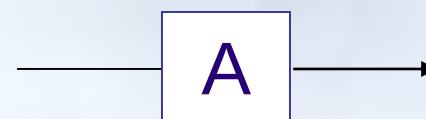
- Với mỗi ký hiệu không kết thúc, lập một sơ đồ cú pháp tương ứng
 - Tên sơ đồ, là tên ký hiệu không kết thúc
- Xây dựng chương trình bởi diễn dịch mỗi sơ đồ cú pháp thành một thủ tục tương ứng
 - Tên thủ tục là tên sơ đồ
 - Mỗi thủ tục có nhiệm vụ triển khai một đoạn nào đó trên văn bản nguồn
- Quá trình phân tích bắt đầu từ thủ tục S và gọi tới các thủ tục khác một cách đệ quy

Xây dựng sơ đồ cú pháp cho các sản xuất EBNF

$\forall a \in V_T$, lập sơ đồ cú pháp

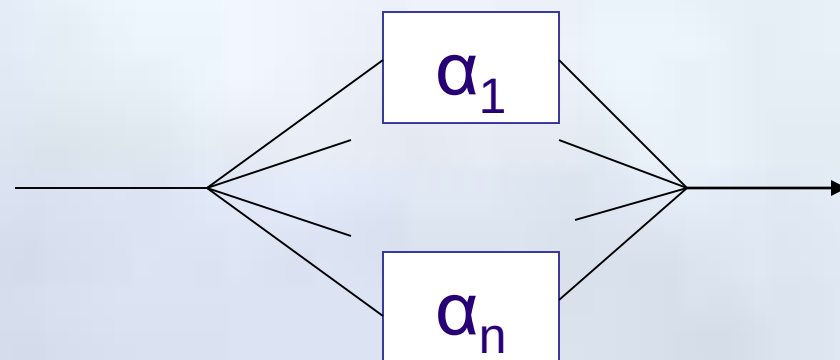


$\forall A \in V_N$, lập sơ đồ cú pháp



$\forall$ sản xuất dạng

$A \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_n$



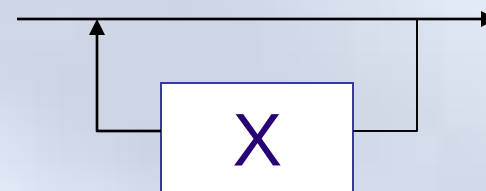
Lập sơ đồ có pháp

Nếu α có dạng $X_1 X_2 \dots X_N$, lập sơ đồ cú pháp



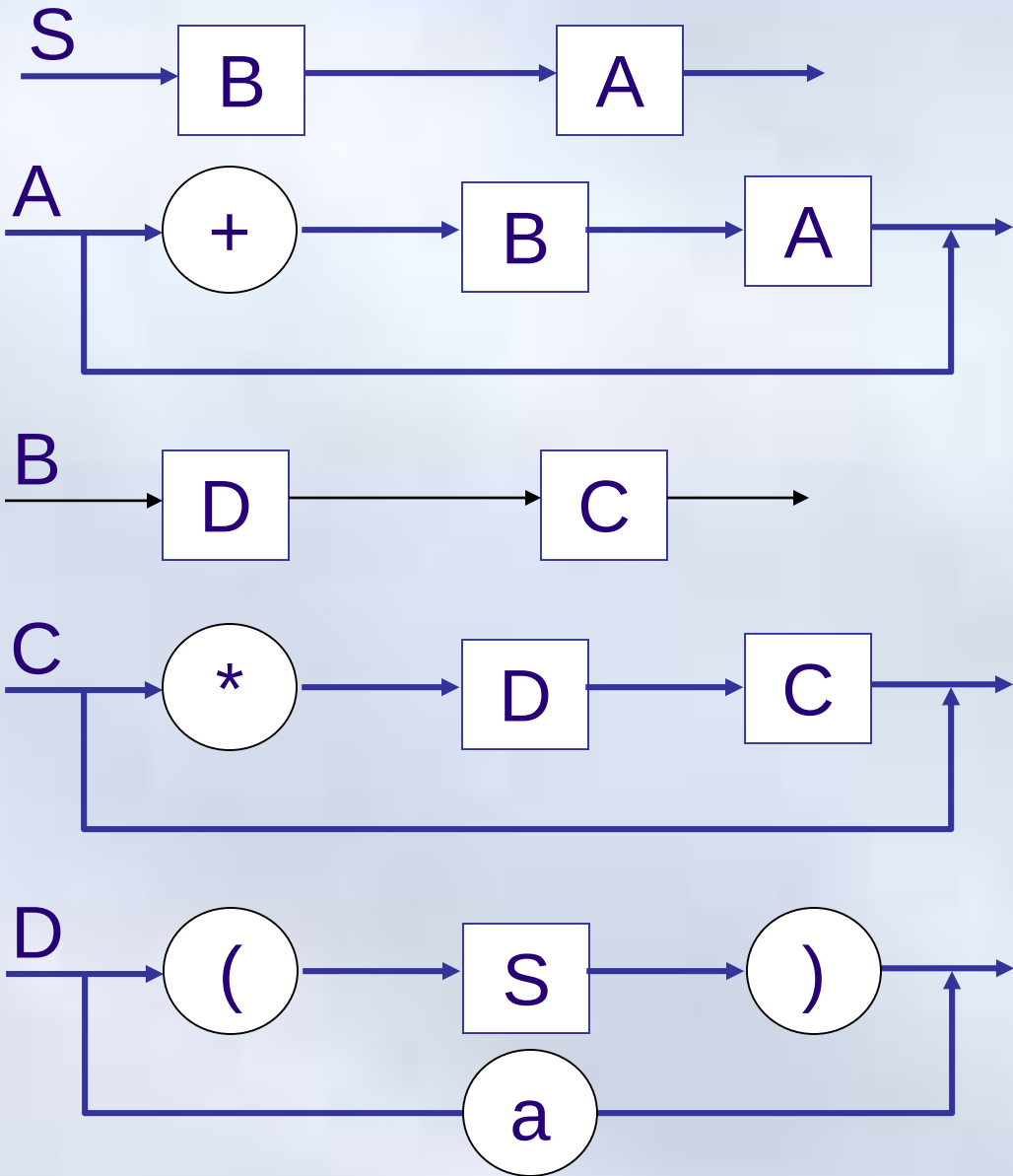
Nếu α có dạng lặp $\alpha \equiv \{X\}$

Sơ đồ cú pháp có dạng



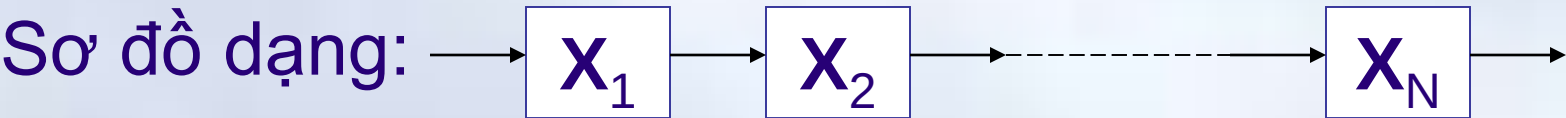
Xây dựng sơ đồ cú pháp → Ví dụ

$$S \rightarrow BA$$
$$A \rightarrow +BA | \epsilon$$
$$B \rightarrow DC$$
$$C \rightarrow *DC | \epsilon$$
$$D \rightarrow (S) | a$$

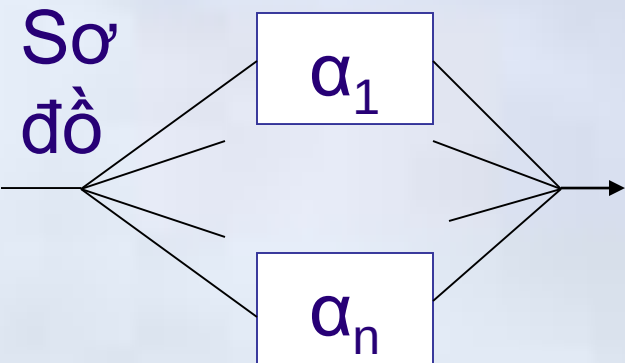


Xây dựng chương trình

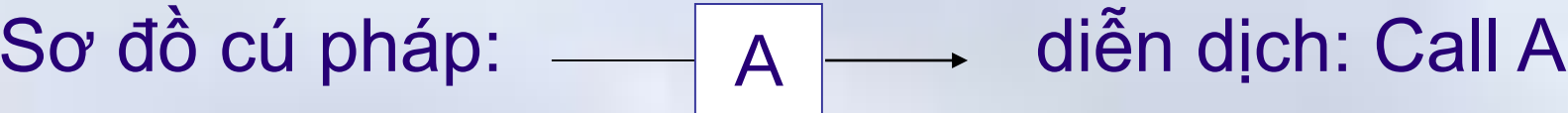
- $T(S)$ là câu lệnh diễn dịch cho sơ đồ S
- Ch là ký hiệu đọc trước trên xâu vào bởi nextCh



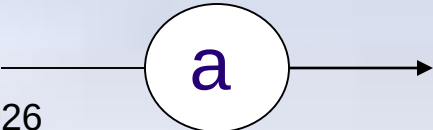
Diễn dịch thành: $\{T(X_1); T(X_2); \dots T(X_N); \}$



```
If Ch in FIRST(alpha_1) Then T(alpha_1)
Else If Ch in FIRST(alpha_1) Then T(alpha_2)
Else If...
Else If Ch in FIRST(alpha_n) Then T(alpha_n)
```

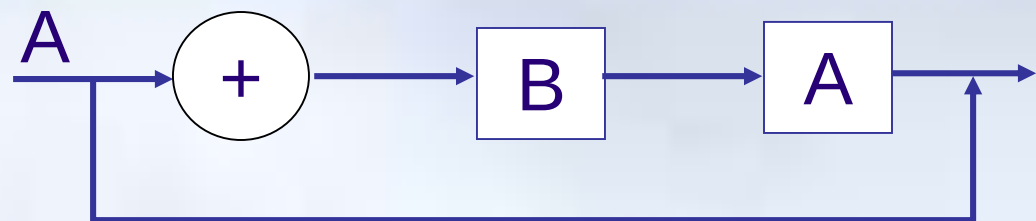


Sơ đồ cú pháp



```
If Ch = a Then nextCh
Else Error (Đang đợi ký hiệu a)
```

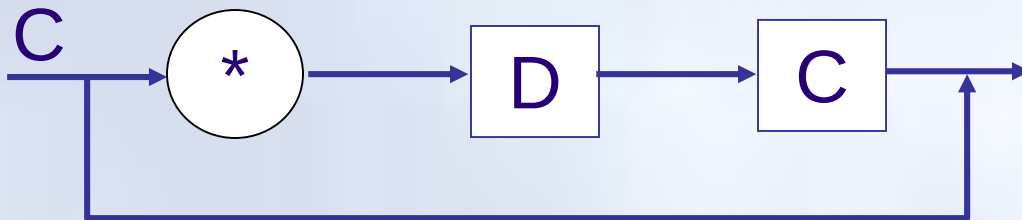
Xây dựng chương trình → Ví dụ



```
void S(){  
    B();  
    A();  
}
```

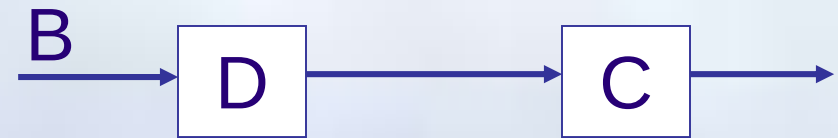
```
void A(){  
    if( Ch=='+' ) {  
        nextCh();  
        B();  
        A();  
    }  
}
```

Xây dựng chương trình→Ví dụ



```

void C(){
    if( Ch=="*" ) {
        nextCh();
        D();
        C();
    }
}
  
```

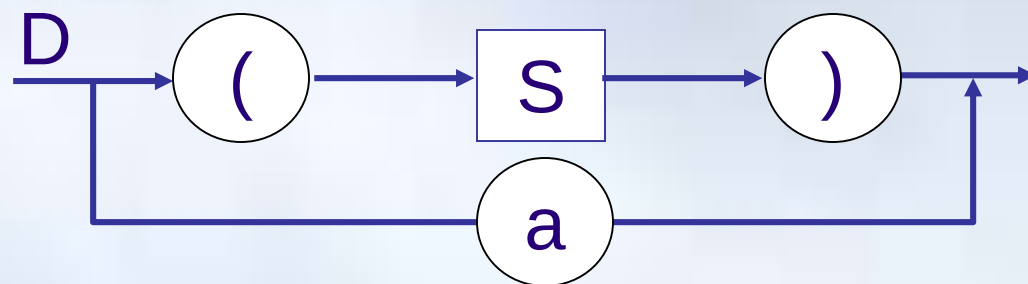


```

void B(){
    D();
    C();
}
  
```

Xây dựng chương trình→Ví dụ

```
void D(){
    if(Ch=='('){
        nextCh();
        S();
        if(Ch==')') nextCh();
        else Error("Thieu đong ngoac");
    } else if(Ch=='a') nextCh();
    else Error("Thieu toan hang");
}
```



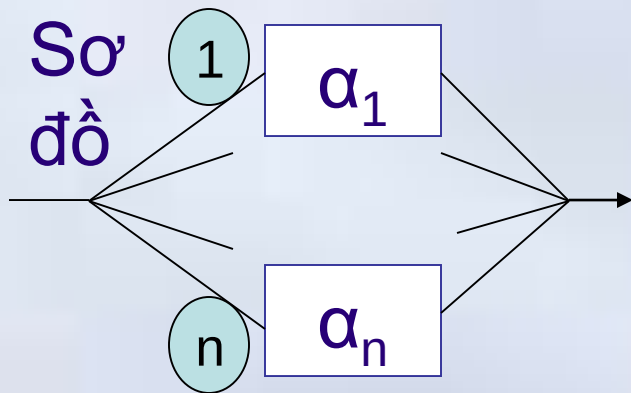
Xây dựng chương trình→Ví dụ

```
void main(){
    gets(Str);    //nhập xâu cần phân tích
    nextCh();     //đọc trước một ký hiệu
    S();          //bắt đầu phân tích
    printf("Bieu thuc hop le...\n");
}
```

$(a+a)*a$	→ Bieu thuc hợp lệ
$a*a+$	→ Thieu toan hang
$a+*a$	→ Thieu toan hang
$a*(a+a$	→ Thieu dong ngoac
$a+aa$	→ Bieu thuc hợp lệ

Tìm danh sách sản xuất sử dụng

- Các nhánh trên sơ đồ được đánh số
 - Nhánh sơ đồ ứng với số hiệu sản xuất
- Khi đi theo một nhánh, cần viết ra số hiệu nhánh trước khi thực hiện câu lệnh diễn giải

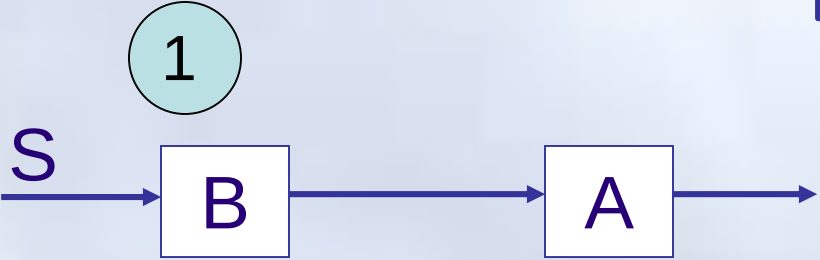


If Ch in FIRST(α_1) Then
 Print("1"); T(α_1)

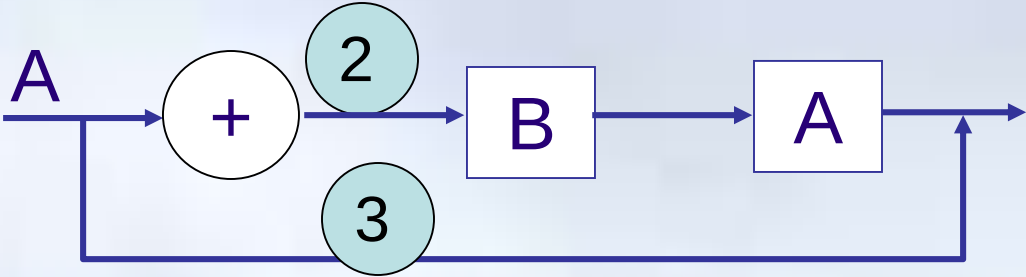
Else If...

Else If Ch in FIRST(α_n) Then
 Print("n"); T(α_n)

Tìm danh sách sản xuất sử dụng → Ví dụ



```
void S(){  
    sx[n++]='1';  
    B();  
    A();  
}
```



```
void A(){  
    if( Ch=='+' ) {  
        sx[n++]='2';  
        nextCh();  
        B();  
        A();  
    }else sx[n++]='3';  
}
```

Chương 3: Phân tích cú pháp

1. Giới thiệu
2. Phương pháp phân tích cú pháp quay lui
3. Phương pháp phân tích bảng
4. Phương pháp phân tích cú pháp tất định
5. Xây dựng bộ phân tích cú pháp cho PL/0

Nguyên tắc

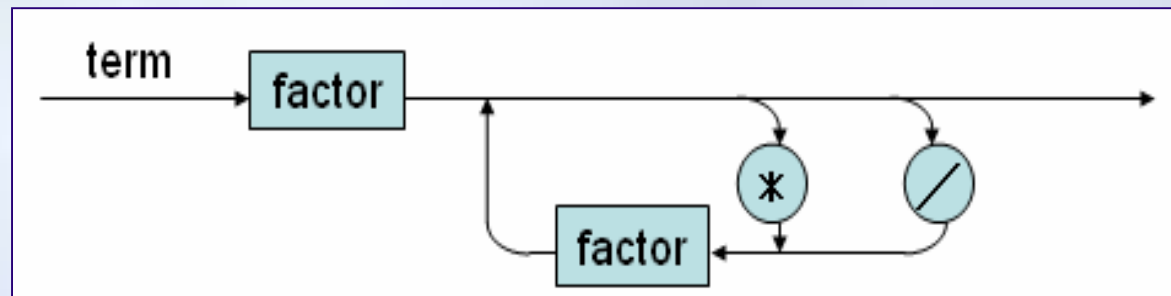
- Văn phạm của PL/0 là LL(1)
 - Sử dụng phương pháp phân tích xem trước
 - Bảng phân tích lớn
 - Dùng phương pháp đệ quy trên xuống
- Bộ phân tích cú pháp
 - Gồm tập thủ tục, mỗi thủ tục ứng với một sơ đồ
 - Luôn đọc trước một ký hiệu/từ tổ
 - Xem trước một từ tổ sẽ cho phép chọn đúng đường đi khi gặp điểm rẽ nhánh trên sơ đồ cú pháp
 - Khi thoát khỏi thủ tục triển khai một đích, luôn có một từ tổ đã được đọc dôi ra

Các thủ tục

- `Token * getToken()` //phân tích từ vựng
- `void error (const char msg[]);`//Báo lỗi
- `void factor(void);`//phân tích nhân tử
- `void term(void);`//phân tích số hạng
- `void expression(void);` // phân tích biểu thức
- `void condition(void);` // phân tích điều kiện
- `void statement(void);` // phân tích câu lệnh
- `void block(void);` // phân tích các khối câu lệnh
- `void program();` //Phân tích chương trình

Term

```
void term()
{
    factor();
    while(Token == TIMES || Token == SLASH)
    {
        Token = getToken();
        factor();
    }
}
```

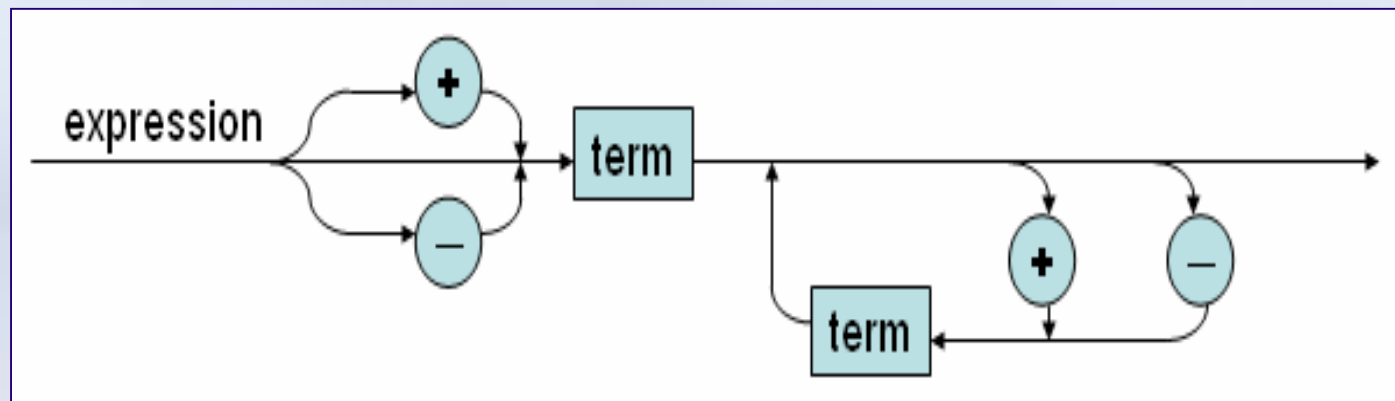


Expression

```

void expression() {
    if(Token == plus || Token == minus)
        Token = getToken();
    term();
    while(Token == plus || Token == minus){
        Token = getToken();
        term();
    }
}

```

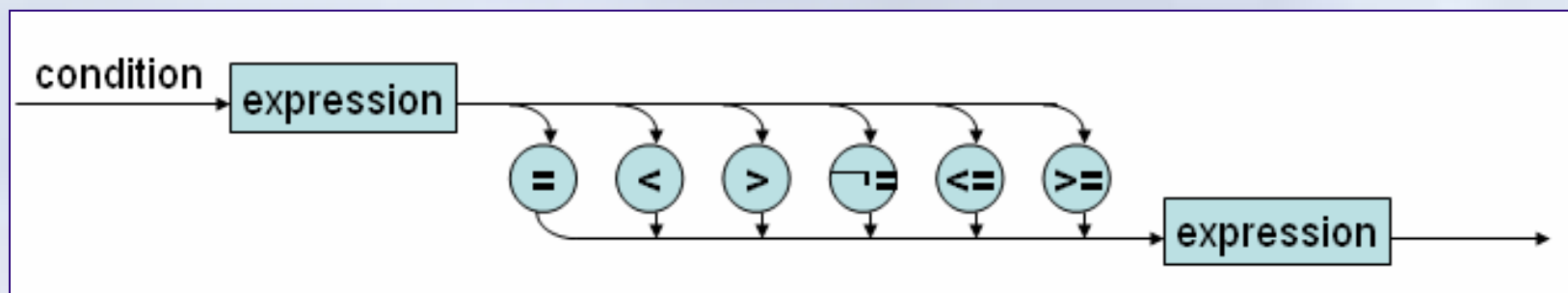


Condition

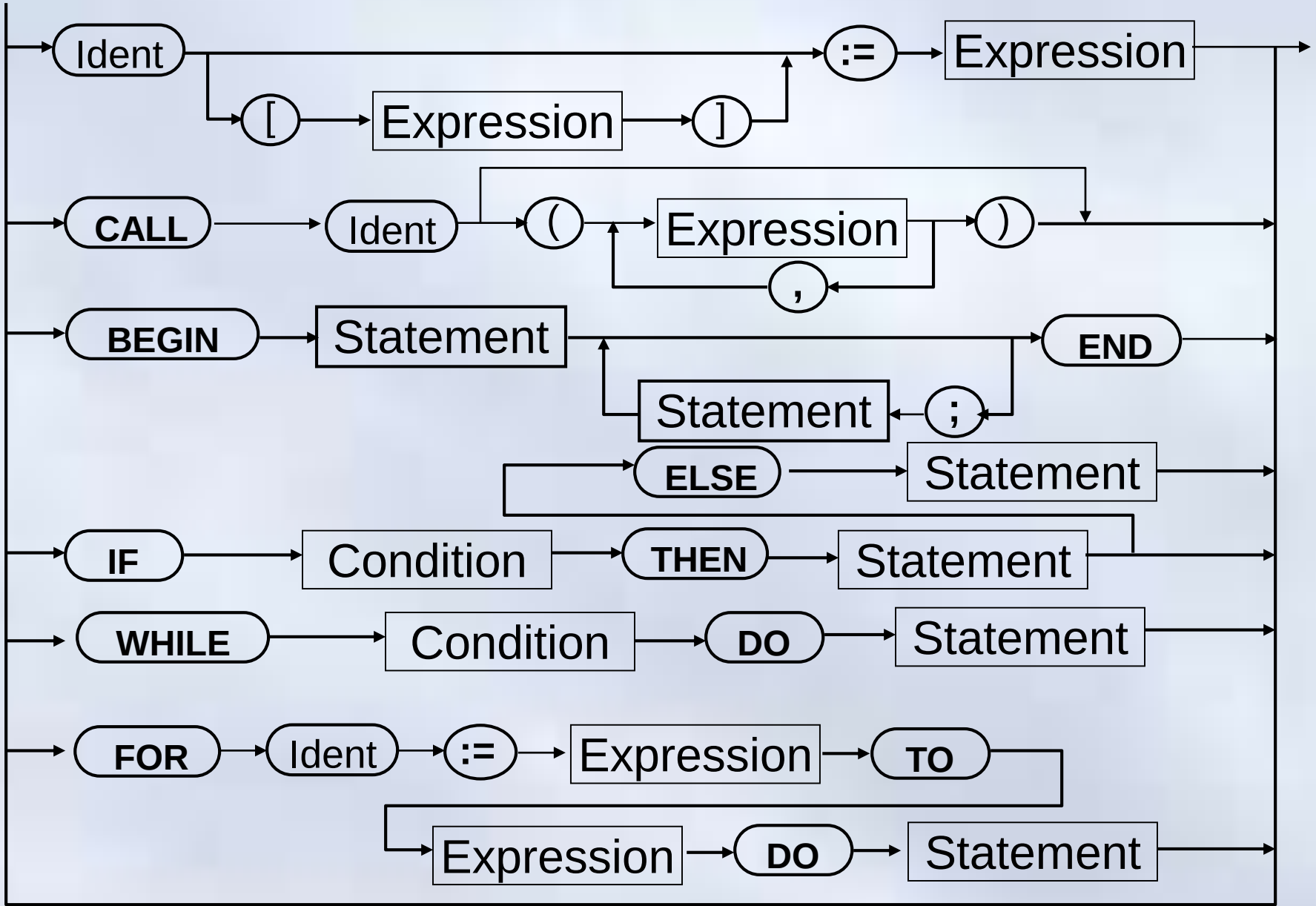
```

void condition() {
    expression();
    if(Token == eql || Token == neq || Token == lss ||
        Token == leq || Token == grt || Token == geq){
        Token = getToken();
        expression();
    }else {
        error("condition: syntax error");
    }
}

```



Statement



Statement

```

void statement() {
    if(Token==Ident){    //variable := <expression>
        Token =getToken();    //array variable ?
        if(Token==assign){
            getToken();
            Expresion();
        }else Error("Thieu toan tu gan")
    }
    else.....
}

```

Statement

```
if (Token == CALL){  
    Token= getToken();  
    if(Token == IDENT){  
        if(Token == LPAREN){  
            Token = getToken();  
            Expression();  
            while(Token==COMMA){  
                Token = getToken();  
                Expression();  
            }  
            if(Token = RPAREN) getToken()  
            else Error("Thiếu dấu đóng ngoặc");  
        }  
    }else Error("Thiếu tên thủ tục/hàm");  
}else...
```

Statement

```
if (Token == BEGIN){  
    Token = getToken()  
    Statement();  
while (Token == SEMICOLON) {  
    Token = getToken();  
    Statement();  
}  
if(Token==END) Token = getToken()  
else Error("Thiếu từ khóa End");  
}else...
```

Program

```
void Program(){  
    if(Token=PROGRAM){  
        Token = getToken();  
        if (Token==IDENT){  
            getToken();  
            if(Token==SEMICOLON){  
                getToken();  
                Block();  
                if(Token==PERIOD)  
                    printf("Thành công");  
                else Error("Thiếu dấu .");  
            }else Error("Thiếu dấu chấm phẩy");  
        }else Error("Thiếu tên chương trình");  
    }else Error(Thiếu từ khóa Program)
```